

先進計算機構成論 12

東京大学大学院 情報理工学系研究科 創造情報学専攻

塩谷 亮太

shioya@ci.i.u-tokyo.ac.jp

本日の話題 : Graphics Processing Unit (GPU)

■ GPU

- もともとはグラフィックを高速に処理するためにあった

■ GP-GPU (General Purpose computing on GPU)

- グラフィック以外に、汎用の計算に GPU を使用する使い方
 - 大量の単純な処理の繰り返しが得意
 - 行列積 → 機械学習, 科学技術計算
 - 仮想通貨のマイニング
- ## ■ この講義では GP-GPU での使用を前提として説明
- 以降で「GPU」と言った場合, GP-GPU での使用を仮定
 - グラフィックのための機能には基本触れない

目標

- 目的：以下を大ざっぱにでも理解する
 - GPU は CPU とは何が違うのか？
 - なぜ GPU やアクセラレータは、AI で高速で効率が良いのか？
- 「GPU は CPU より単純な小さいコアをよりたくさん積んでいるため性能が高い」
 - しかし…
 - なぜ GPU では小さく単純にできる？
 - そもそも「小さい」とか「単純」とはどういうこと？
- これらの疑問に答えられるようにしたい

もくじ

1. これまでの振り返りとエネルギー効率
 1. CPU の動作
 2. 命令パイプライン
 3. CMOS 回路の消費エネルギー
2. GPU の基本的な構造とプログラミング・モデル
3. (AI) アクセラレータ

これまでの振り返りとエネルギー効率の定義

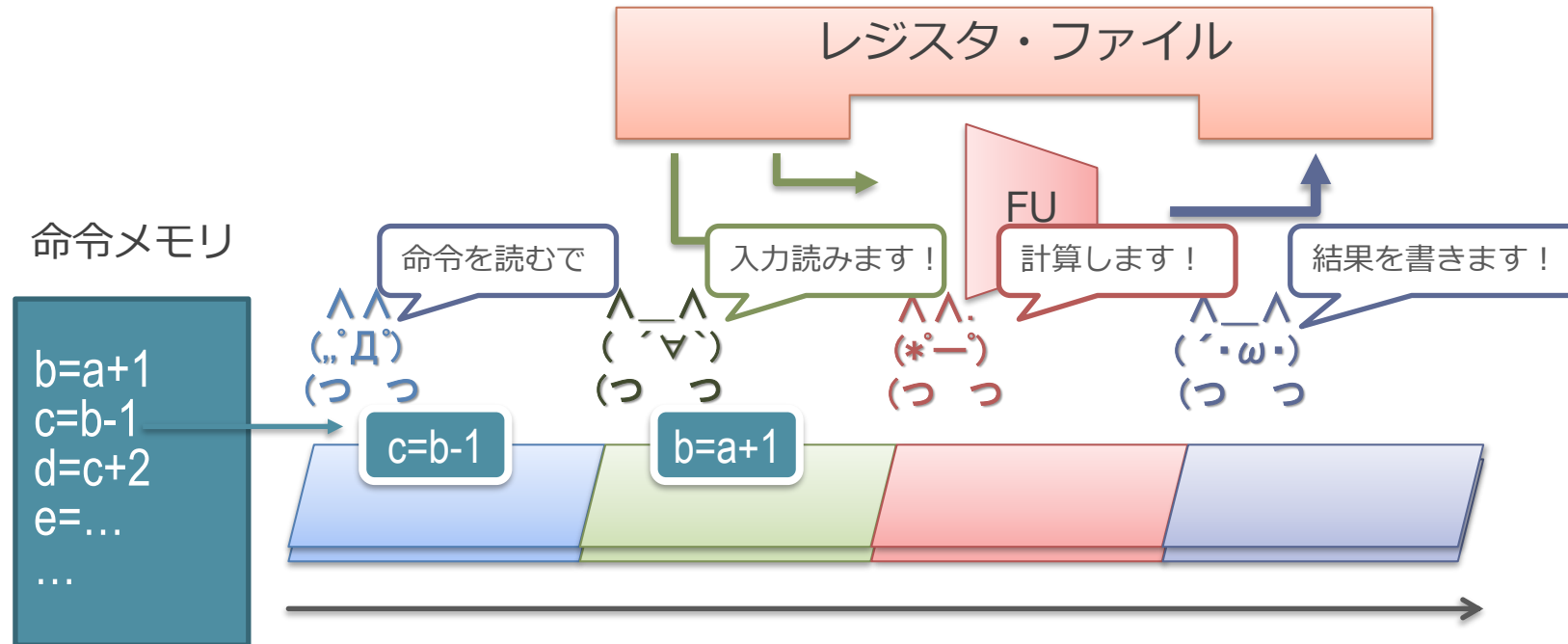
CPU の動作は料理に似ている（第1回）

- レシピをみながら料理をするのに似ている
 - レシピの各手順が命令
 - 「今何個目の手順を見てるか」，を憶えているのが PC
 - ひとつひとつ手順を取り出して，指示に従って処理

カレーのレシピ

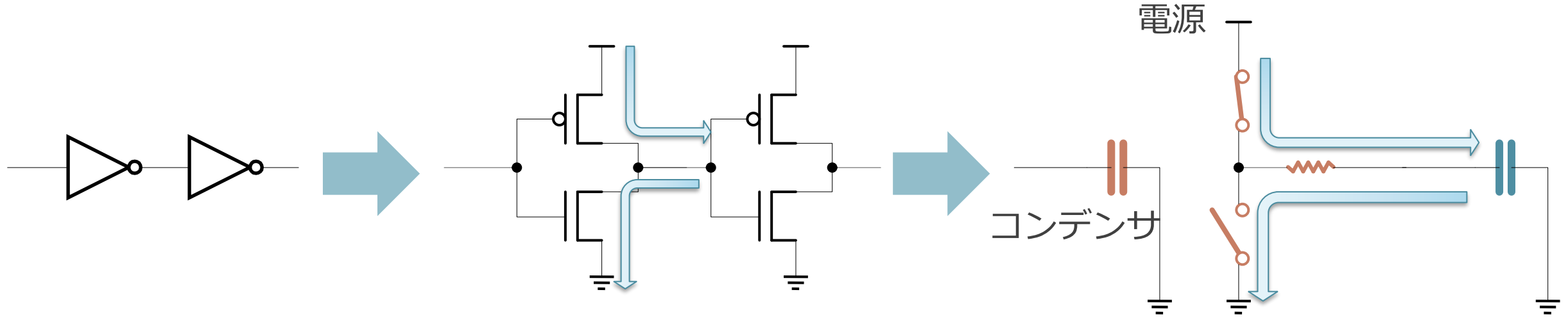


命令パイプライン（第3回）



- $(\text{,,}\Delta)$ が命令をメモリから読み出す
- $(\text{'}\nabla\text{'})$ がレジスタ・ファイルから入力の値を読み出す
- $(\text{*}\text{--})$ が演算器（FU）で計算する
- $(\text{'}\omega\text{'})$ がレジスタ・ファイルに結果を書き込む

CMOS 回路におけるスイッチングの消費エネルギー（第2回）

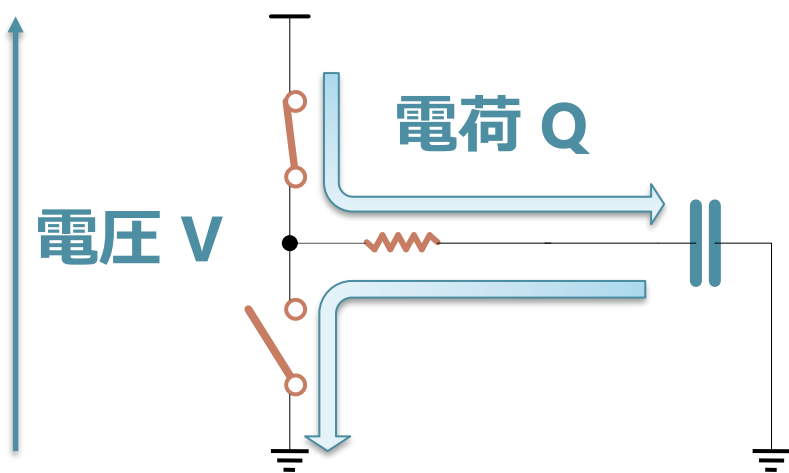


- CMOS 回路は、コンデンサに連動したスイッチとみなせる
（MOS 構造のゲートのところがコンデンサになってる）
 - コンデンサが充電状態：下のスイッチがON
 - コンデンサが放電状態：上のスイッチがON
- 主に計算で消費されるエネルギー：
 - スイッチを ON/OFF するためのコンデンサへの充放電で消費

これも水流で考える

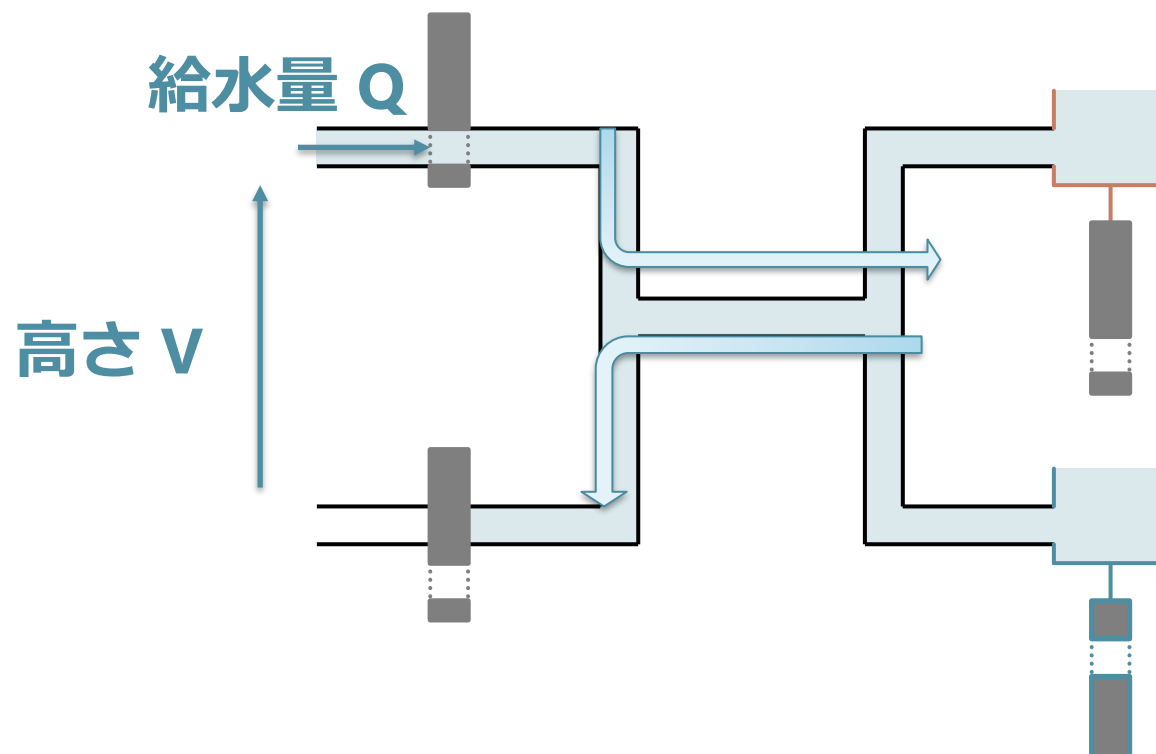
■ 電流を流して失われるエネルギー

- コンデンサに溜まる
電荷の量 : Q
- 電圧 : V
- 消費エネルギー : $Q \times V$



■ 水を高いところから流すとその分位置エネルギーが失われる

- 水の総量 : Q
- 排水口から給水口高さ : V
- エネルギー : $Q \times V$



振り返りからわかること

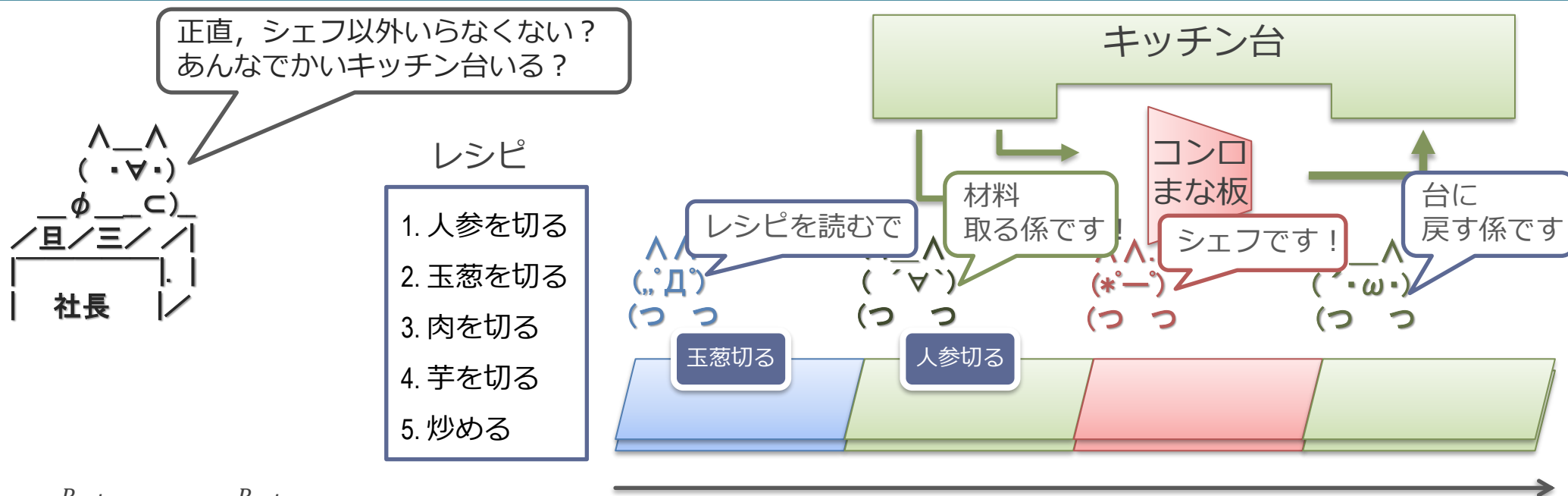
1. CPU の動作はカレーを作るのに似ている（第 1 回, 第 3 回）
 - 命令を読んで, レジスタを読んで, 計算し, レジスタに戻す
 - 流れ作業によって効率を上げている
 2. 消費電力は回路の大きさに比例する（第 2 回）
 - 主に計算で消費されるエネルギー：
 - スイッチを ON/OFF するためのコンデンサへの充放電で消費
 - 回路の大きさ→コンデンサの総容量
 - 水タンクの総量が大きくなるイメージ
- これらをもとに, エネルギー効率とアーキテクチャを見ていく

CPU や GPU などの「エネルギー効率」とは

牧野先生の Principles of High-Performance Processor Design より

- 効率 $\eta = \frac{P_{min}}{P_{actual}} = \frac{P_{min}}{P_{min} + P_{overhead}}$
- P_{min} = そのアプリケーション内の「演算」を行うのに最低必要なエネルギー
 - たとえば行列積であれば、乗算や加算そのものに必要なエネルギー
- P_{actual} = 実際にチップで消費されたエネルギー
 - 命令を読み出したり、レジスタを読み書きしたりなどのに必要な追加のエネルギー $P_{overhead}$ がのってくる
- 「エネルギー効率」を上げるのに取り得る方針：
 - $P_{overhead}$ を減らす
 - (全体のエネルギーを減らすなら、 P_{min} 自体を減らす方向もある (付録に記載))

カレー屋さんパイプラインで考える



■ 効率 $\eta = \frac{P_{min}}{P_{actual}} = \frac{P_{min}}{P_{min} + P_{overhead}}$

- P_{min} = 材料の原価と「シェフ」の人件費（絶対にカレーを作るのに必要）
- $P_{overhead}$ = キッチン台の設備費や他の係の人の人件費（必ずしもなくてもよい）

■ 効率を上げるのに取り得る方針：

- $P_{overhead}$ を減らす = 設備を減らす, シェフ以外をなんとかしてクビにするか減らす（こっちの話をします）
- (P_{min} を減らす = 別の方法としてカレー自体の具材を減らして質を下げると, 効率 η は上がらないがトータルコストは減る

GPU のアーキテクチャ

GPU が対象とする処理

- 大量の単純な処理の繰り返し
 - 並列処理できる部分が容易にわかる
 - 典型的な例：行列積

並列処理できる部分が自明なループの例

■ ループで書いた行列積のコード

```
for (k = 0; k < 1024; k++) // 最外周ループ
    for (j = 0; j < 1024; j++)
        for (i = 0; i < 1024; i++)
            a[j][i] += b[j][k] * c[k][i];
```

■ ループ内の乗算や加算は、明らかに並列にできる

- ループの各周回の乗算を並列にやって、別々に足し合わせていける
- (浮動小数点の場合、厳密には多少結果が変わる)

対象とする処理の違い

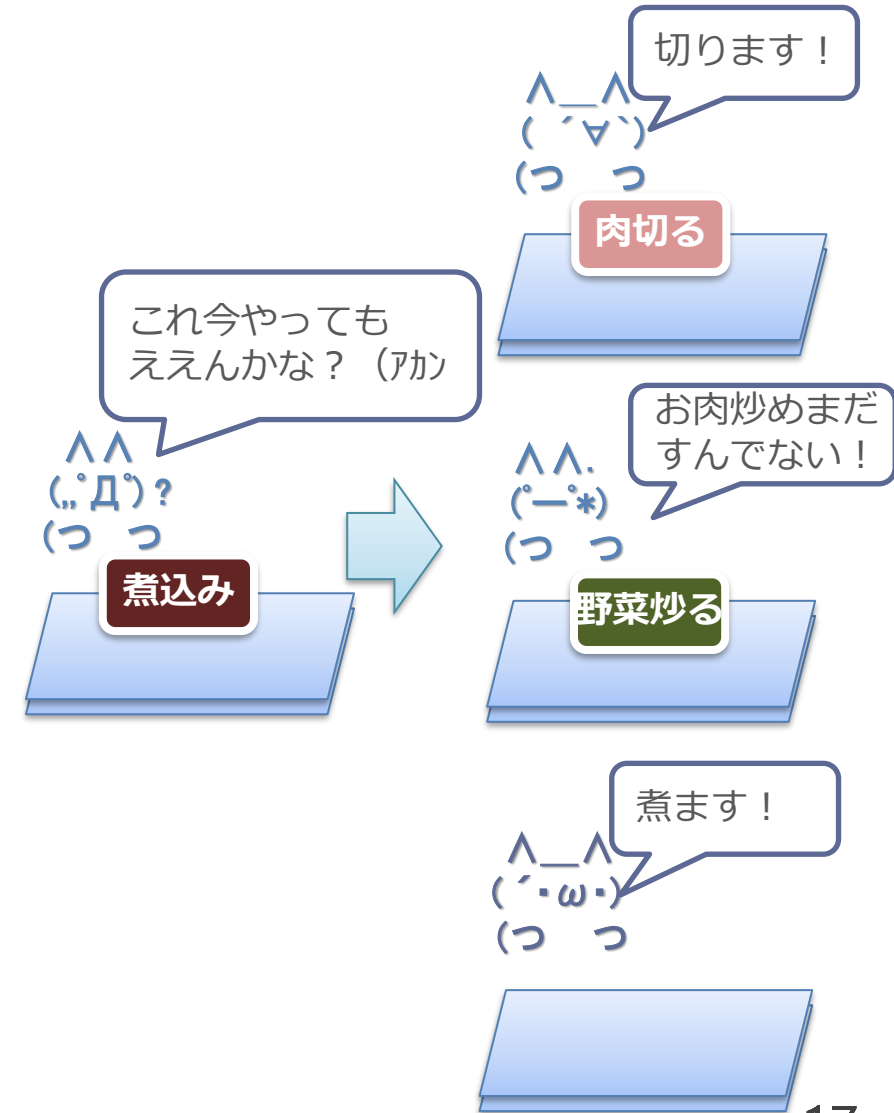
- CPU :
 - 単一の連続した処理
- GPU :
 - 大量の単純な処理の繰り返し
- ここではカレー屋さんで説明

単一の連続した処理= 1つのカレーを速く作る場合

- 1つのカレーを並列処理で高速に作るのは難しい
 - 「2.野菜を炒める」は「1.野菜を切る」を先にやる必要がある
 - 「2.野菜を炒める」と「3.肉を切る」は並列にできる
- 先頭の人レシピを見極めて仕事を割り振ればある程度できなくはない
 - 右側に3倍の人員を割いているが、3倍の速さにはならない
 - 依存があるので、先に並列処理できないことが多い
- OoO スーパスカラプロセッサはまさに上記のよう

カレーのレシピ

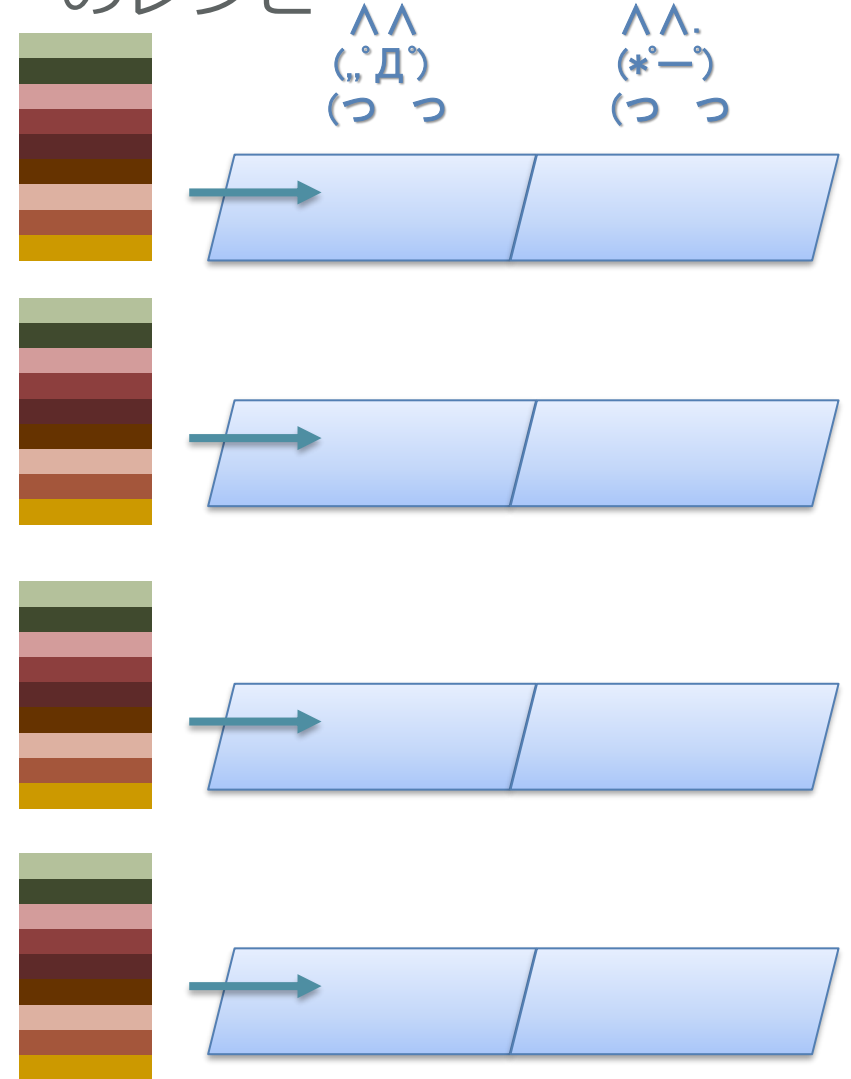
- 1.野菜を切る
- 2.野菜を炒める
- 3.肉を切る
- 4.肉を炒める
- 5.煮込み
- 6.ルーを入れる
- 7.コメを研ぐ
- 8.コメを炊く
- 9.盛りつける



大量の単純な処理の繰り返し = カレーをたくさん作る場合

- 単に並べてみんなで同じ処理をすれば速くなる
 - ループの各周をみんなでやってくださいと言うだけ
 - 振り分けとか考える必要もない
- 並べた分だけ速くなる（すごい）
 - 右だと4倍速い
- GPU はそう言うことが出来るプログラムがターゲット
 - もともとグラフィック処理では、画面上の多数のピクセルの処理がこれに該当
 - ピクセル間の処理は基本的に依存がない
 - 画面の上の方と下の方は並列にやっても問題ない

カレーのレシピ



ループのマルチスレッド化による並列実行

- ループで書いた行列積のコード

```
for (k = 0; k < 1024; k++) // 最外周ループ
    for (j = 0; j < 1024; j++)
        for (i = 0; i < 1024; i++)
            a[j][i] += b[j][k] * c[k][i];
```

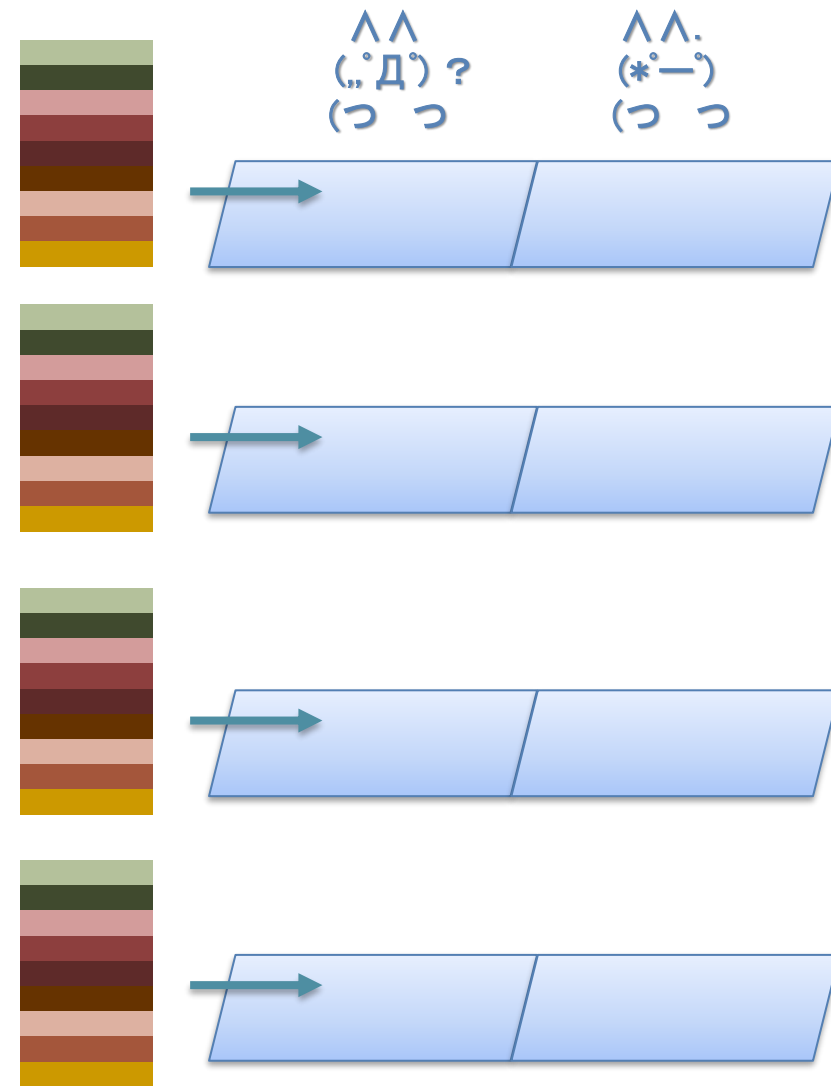
- 最外周ループ部分を複数のスレッドとしてマルチスレッド化

// func が 1024 個起動されて並列に実行される
// 0-1023 がスレッド ID として渡される

```
func(thread_id) {
    k = thread_id;
    for (j = 0; j < 1024; j++) // ここは同じ
        for (i = 0; i < 1024; i++)
            a[j][i] += b[j][k] * c[k][i];
}
```

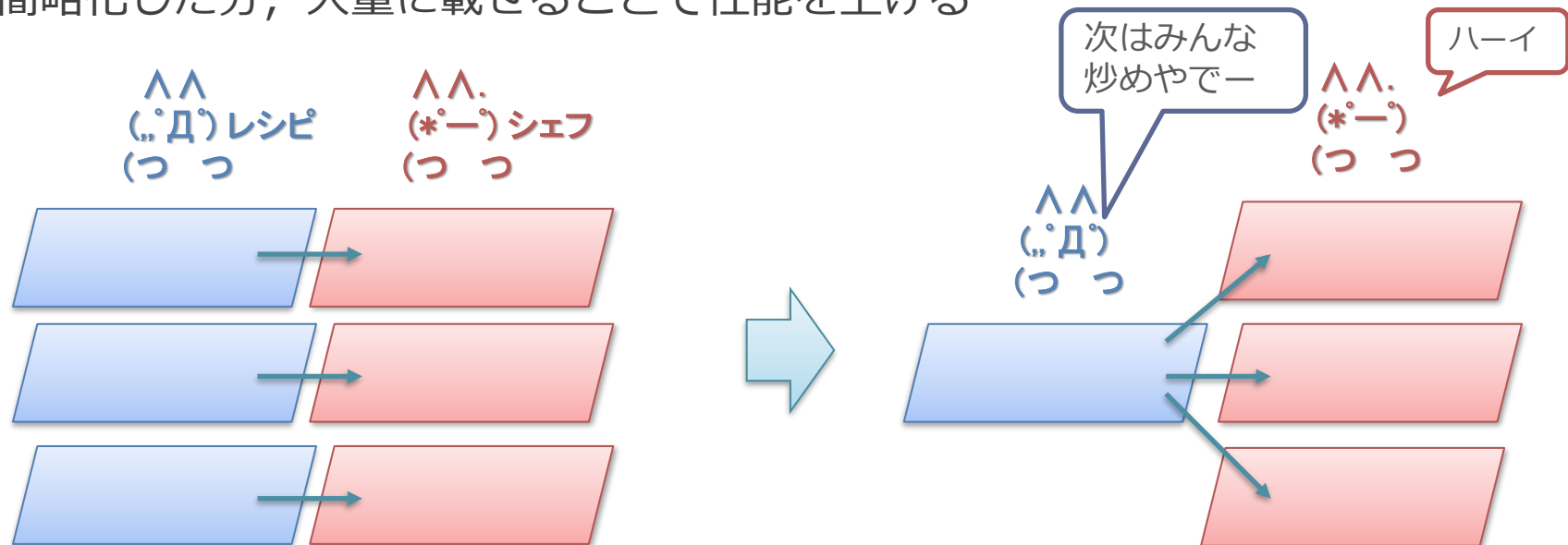
マルチスレッド化による並列実行

- マルチスレッド化による並列実行による高速化
 - 先ほどの例では 1024 スレッドが同時実行される
- マルチコア CPU でも同じなのでは？
 - マルチコア CPU = 複数の CPU を束ねたもの
 - 複数の CPU でそれぞれでスレッドを並列に実行できる



マルチコアとの違い：回路量あたりの性能の向上

- GPU は同じ処理をするスレッドを多数走らせることに特化
 - 行列積の例では，全スレッドが「同じ処理」をしている
 - 正確には，演算（乗算と加算）は同じで，値が異なる
- 方針：みんな同じ処理をしているので，それを利用して回路を簡略化
 - みんな同じ処理をしている = 制御回路を共有できる
 - レシピ自体や読む係は1人で良く，みんなに同じ指示を投げる
 - 簡略化した分，大量に載せることで性能を上げる



代表的な GPU のアーキテクチャ

- GPU は同じ処理をするスレッドを多数走らせることに特化
 - これを効率的に実現するのが SIMD と呼ばれる方式
 - SIMD は GPU のハードウェア方式やプログラミング・モデルの根幹に関わる
- 大きく分けて 2 つの方式
 1. NVIDIA : Single Instruction Multiple Thread (SIMT) 方式
 2. AMD/Intel : Single Instruction stream Multiple Data (SIMD) 方式
- SIMD 方式から順に説明
 - SIMT 方式は SIMD を発展させた概念であるため

GPU の基本的な構造

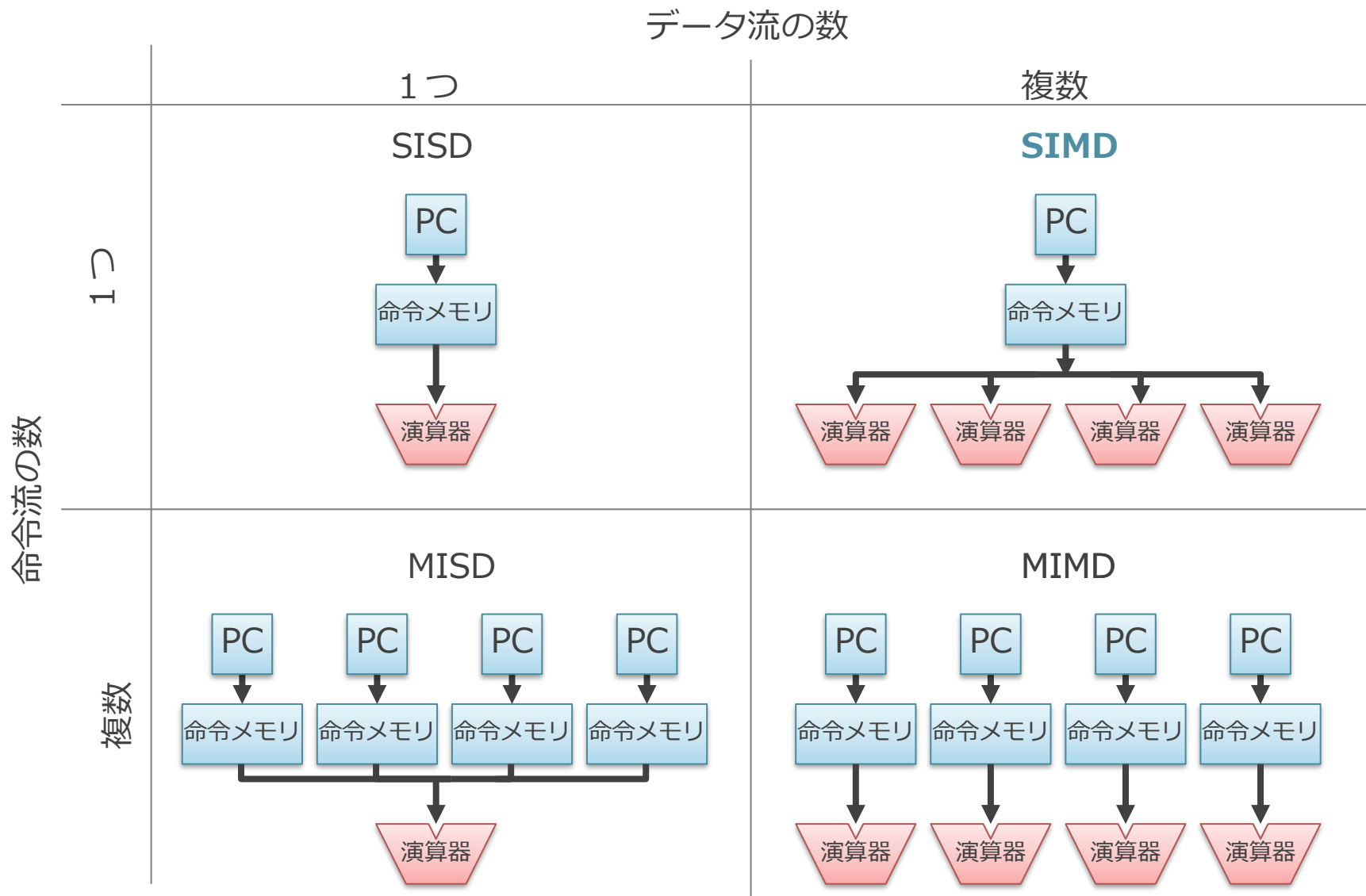
1. フリンの分類と SIMD
2. GPU における回路量当たりの性能向上
3. SIMD プロセッサのプログラミング・モデル

フリンの分類と SIMD

- Single Instruction stream/Multiple Data stream (SIMD)
 - コンピュータ・ハードウェアの構成方法の分類の 1 つ
 - あいだの「stream」は省略されることもある
- フリンによる分類に由来
 - M. J. Flynn, "Very high-speed computing systems," in Proceedings of the IEEE, vol. 54, no. 12, pp. 1901-1909, Dec. 1966
- プログラムを処理する際の
 - 「命令の流れ」と「データの流れ」に着目
 - それぞれが同時に 1 つあるか、複数あるかで分類

フリンの分類

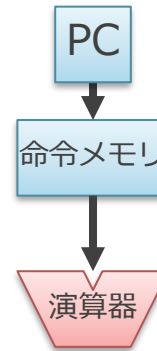
混乱したらこの図を見返そう



■ カレー屋さんとの対応

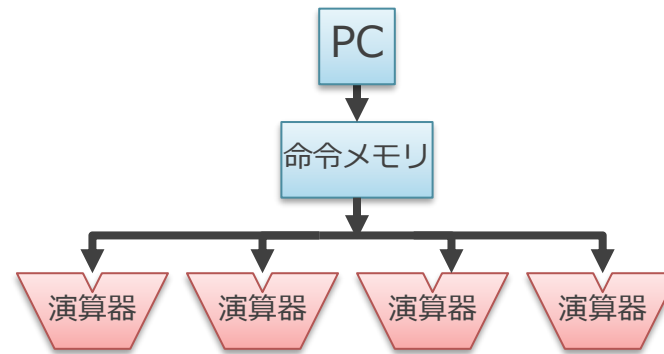
- PC: レシピ上の今やってるところ（しおり）
- 命令メモリ: レシピ
- 演算器: シェフ

SISD (Single Instruction stream/Single Data stream)



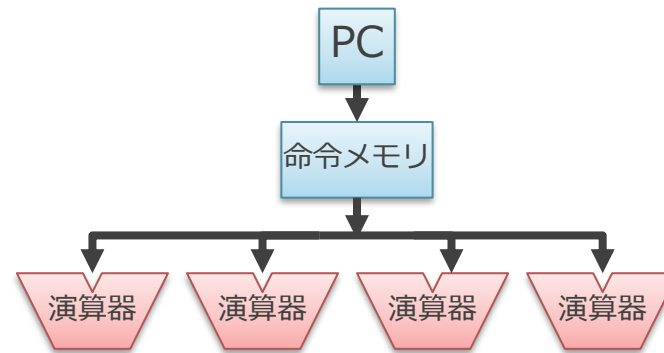
- 1つの命令流が1つのデータ流を処理する方式
 - 動作：
 - 同時に1つのプログラム（命令の流れ）を実行
 - それぞれの命令は1つのデータを演算
 - 典型的にはシングルコア CPU がこれに該当

SIMD (Single Instruction stream/Multiple Data stream)



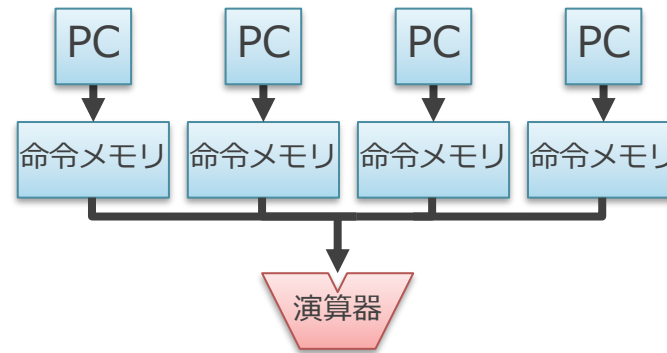
- 1つの命令の流れが複数のデータの流を処理する方式
 - SIMDは同時に1つのプログラム（命令の流れ）を実行
 - それぞれの命令は複数のデータに対して同じ演算を行う
- 例：4つの演算を同時に行う SIMD 方式
 - 解釈された命令は4つの演算器に送信
 - それぞれの演算器は異なる4つのデータに対して同じ演算を行う
 - たとえば加算命令であれば、1つの加算命令が4つの加算を行う

SIMD (Single Instruction stream/Multiple Data stream)



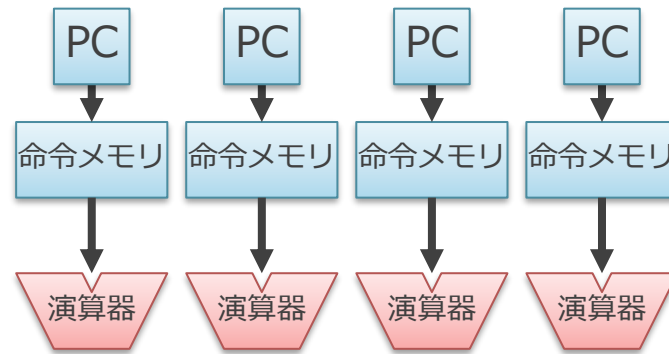
- 現在の主だった GPU はこの SIMD 方式をベースにしている
 - 一部, CPU にもこの考えを取り入れた命令がある
- GPU は同じ処理を多数走らせることに特化
 - PC や命令メモリは共有して, 各演算器に配ればよい

MISD (Multiple Instruction stream/Single Data stream)



- 複数の命令の流れが単一のデータの流を処理する方式
 - 分類の都合上存在していると言ってもよい
 - この方式の実用的なコンピュータは現在一般的ではない
- 「みんなで違うレシピを読んで、1人のシェフに同時に指示を与える」みたいなことに相当し、意味がわからない

MIMD (Multiple Instruction stream/Multiple Data stream)



- 複数の命令の流れが複数のデータの流を処理する方式
 - 典型的にはマルチコア化された CPU がこれに該当

フリンの分類と実際

- 現代のコンピュータをこのフリンの分類にそのまま厳密に当てはめよう
とするとやや無理がある
 - 通常は複数の要素を同時に併せ持つことが多い
- たとえば…
 - GPU は SIMD 方式のコアを複数備えていることが一般的
 - SIMD と MIMD の要素を併せ持つ
 - CPU では一部の命令でのみ複数のデータを同時に処理する
 - SISD と SIMD の要素を併せ持つ
 - マルチコア化されると MIMD の要素もある
- フリンの分類は、あくまでコンピュータをある特定の軸から見た際の分類
であると考える

余談：SIMD の発音

- 業界では「シムディー/sim-dee」と発音する
 - シムド と読んじゃだめ

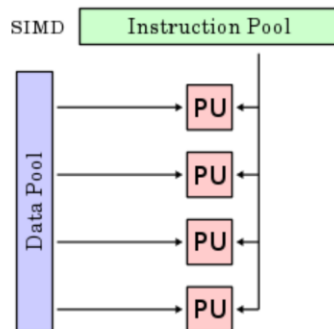
[mdn web docs](#) [References](#) [Guides](#) [MDN Plus](#)

MDN Web Docs Glossary: Definitions of Web-related terms > SIMD



SIMD

Single-Instruction/Multiple-Data Stream
(SIMD or “sim-dee”)



- SIMD computer exploits multiple data streams against a single instruction stream to operations that may be naturally parallelized, e.g., Intel SIMD instruction extensions or NVIDIA Graphics Processing Unit (GPU)

SIMD (pronounced “sim-dee”) is short for **Single Instruction/Multiple Data** which is one [classification of computer architectures](#). SIMD allows one same operation to be performed on multiple data points resulting in data level parallelism and thus performance gains — for example, for 3D graphics and video processing, physics simulations or cryptography, and other domains.

それぞれ以下より

Mozilla の開発者ページ

<https://developer.mozilla.org/en-US/docs/Glossary/SIMD>

UCSB の講義資料

<https://sites.cs.ucsb.edu/~tyang/class/240a17/slides/SIMD.pdf>

イラスト

村上太一@TIER IV

もくじ

1. フリンの分類と SIMD
2. GPU における回路量当たりの性能向上
 1. 演算器と制御部
 2. SIMD による制御部の共有
 3. 制御部自体の小型化
3. SIMD プロセッサのプログラミング・モデル

回路量当たりの性能の背景：演算器と制御部

■ CPU や GPU などを構成する回路を以下に分けて考える

● 制御部（青）

- 命令を解釈しそれに基づいて演算器を制御する回路
- 右の図だと PC や命令メモリ

● 演算器（赤）

- 加算や乗算などの演算そのものを行うための回路

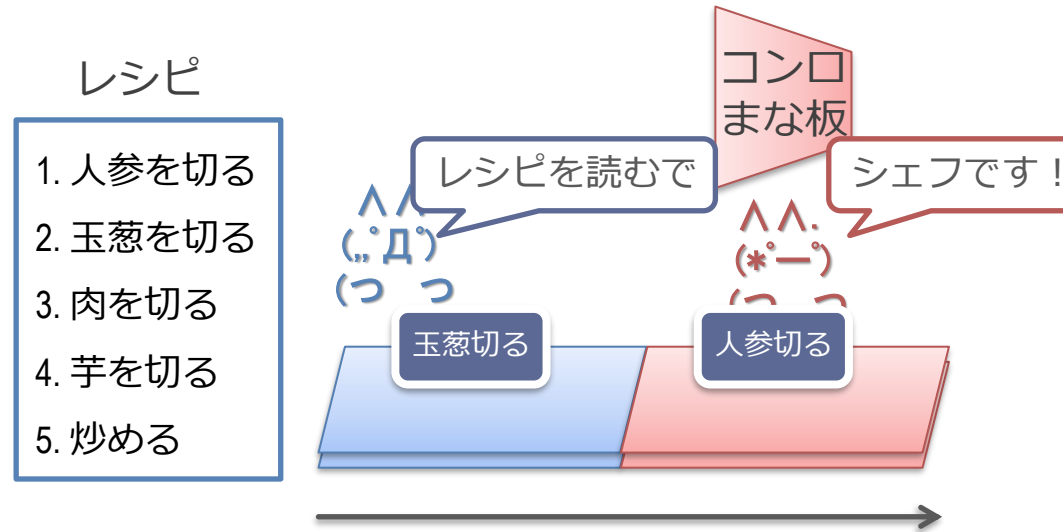


■ これに基づいて回路量あたりの性能を議論

■ ポイント：演算器部分自体は CPU でも GPU でも基本的に変わらない

- GPU が効率が良い演算器を持っているなら、それは CPU にも持っていけるはず
 - AI に特化した演算器は CPU にも取り入れられている
- それ以外をどう減らすかが要点

またカレー屋さんで考えていく

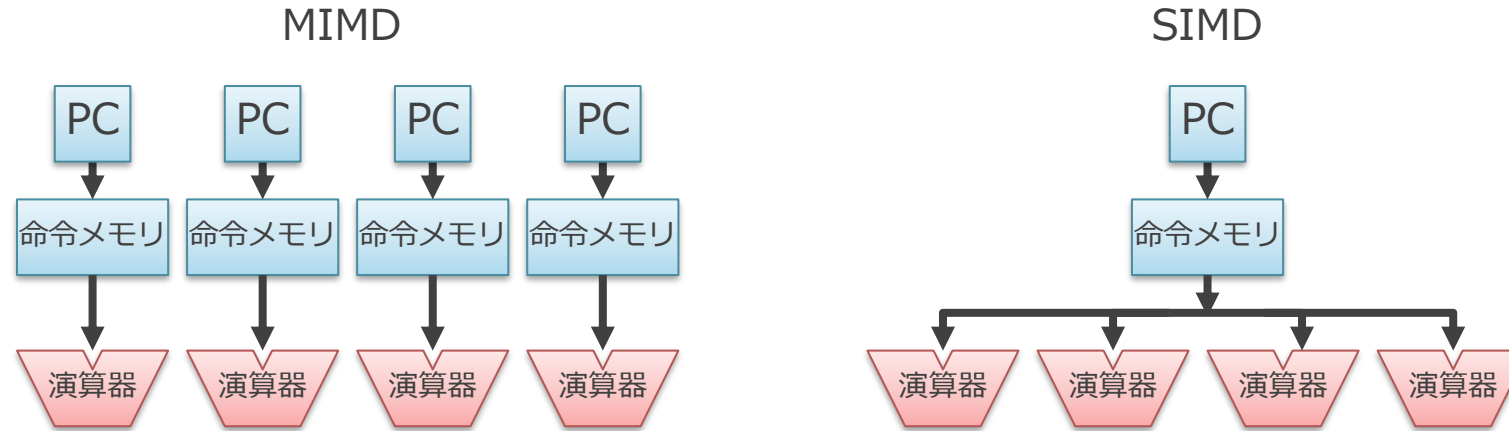


- 制御部（青）：PC と命令メモリ
 - レシピと読む係の人
- 演算器（赤）：そこより後ろで実際に乗算や加算をしている部分
 - シェフの人
 - キッチン台や関連するスタッフは一旦ここでは忘れる
（まだクビになったわけではないが、後で合理化のためにリストラする予定）

GPU における回路量あたりの性能向上

1. 制御部の共有
2. 制御部の小型化

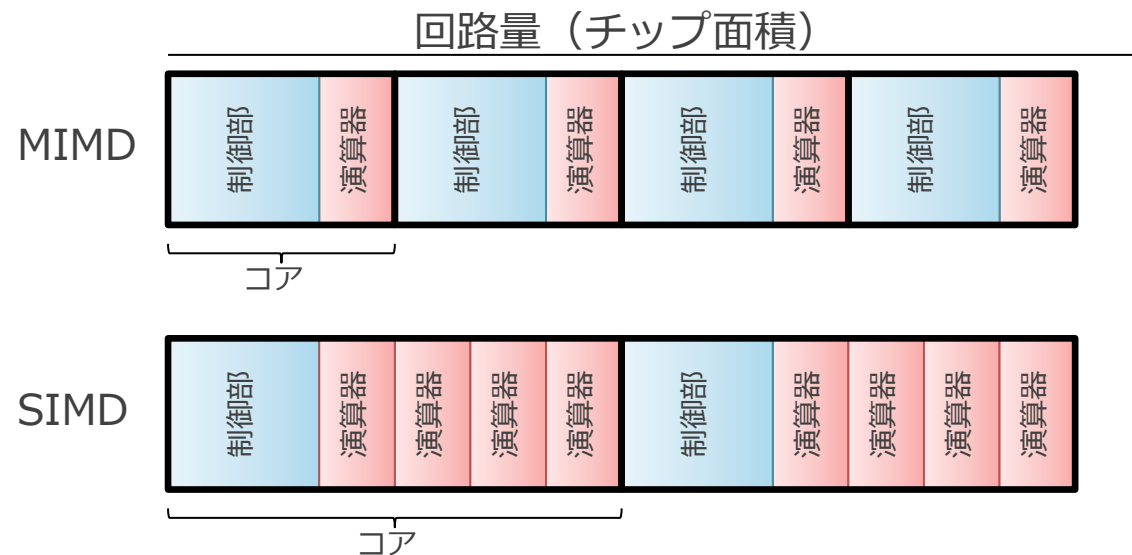
SIMD では複数の演算器間で制御部を共有できる



- たとえば, 上の図の MIMD と SIMD の場合
 - 4つの演算器を持ち最大で同時に4並列で演算ができる
 - = 同じ最大性能を持つ
 - SIMD では 1つの制御部が演算器間で共有されている
 - 実際の GPU では 16 個程度の演算器が共有されていることが多い

SIMD は同じ回路量で高い性能を実現できる

- SIMD は MIMD と比べて全体をより小さく作ることができる
 - 同じ総面積であれば、SIMD の方がより多くの演算器を搭載できる
 - 下の図だと演算器は **MIMD 4 個** vs. **SIMD 8 個**
 - つまり、同じ回路資源量あたりで、高い性能を出せる
 - 払える人件費が一定の場合、シェフ 4 人あたりレシピ読み係を 1 人にして、シェフの人数を増やす事に相当
- これが、どの GPU も基本的には SIMD を採用している理由



GPU における回路量あたりの性能向上

1. 制御部の共有

2. 制御部自体の小型化

- （この話は SIMD の話とは直交しているが、関わるのでここで

GPU では制御部自体を簡素化

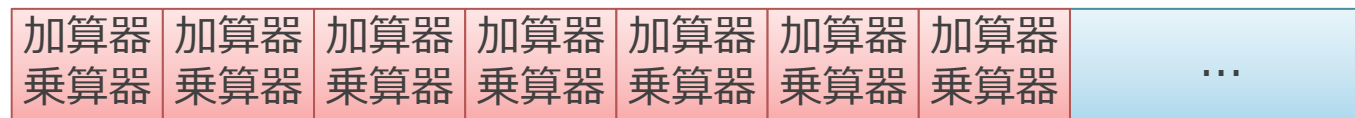
■ CPU

- 単一のスレッドの実行時間を短くすることに特化
- 各種投機実行や、動的命令スケジューリングに回路資源を投入



■ GPU

- 大量のスレッドの実行スループットを上げることに特化
- 演算器以外の部分をそぎ落とし、なるべく大量の演算器を積む

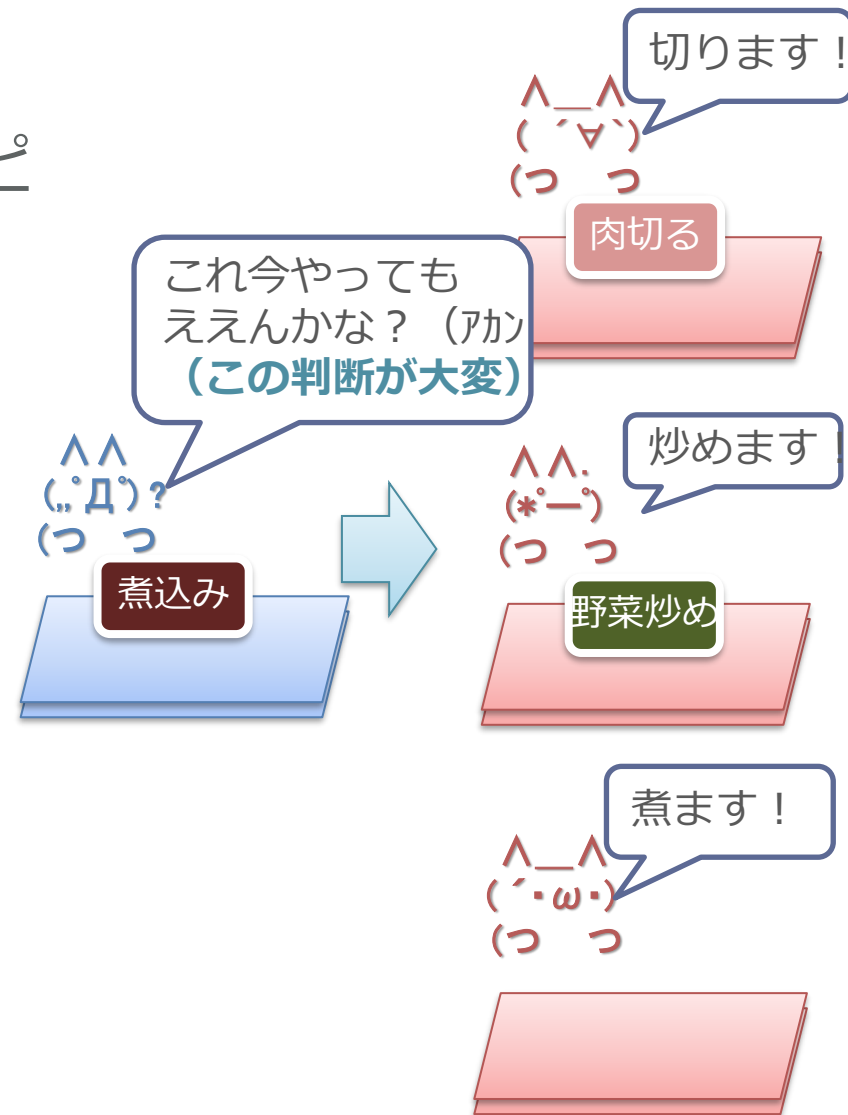


仕事の割り振りは超大変

- 並列にできる部分を探す回路は、演算そのものよりも遥かに複雑
- 現代の高性能 CPU は、ここに大半の回路を割いている

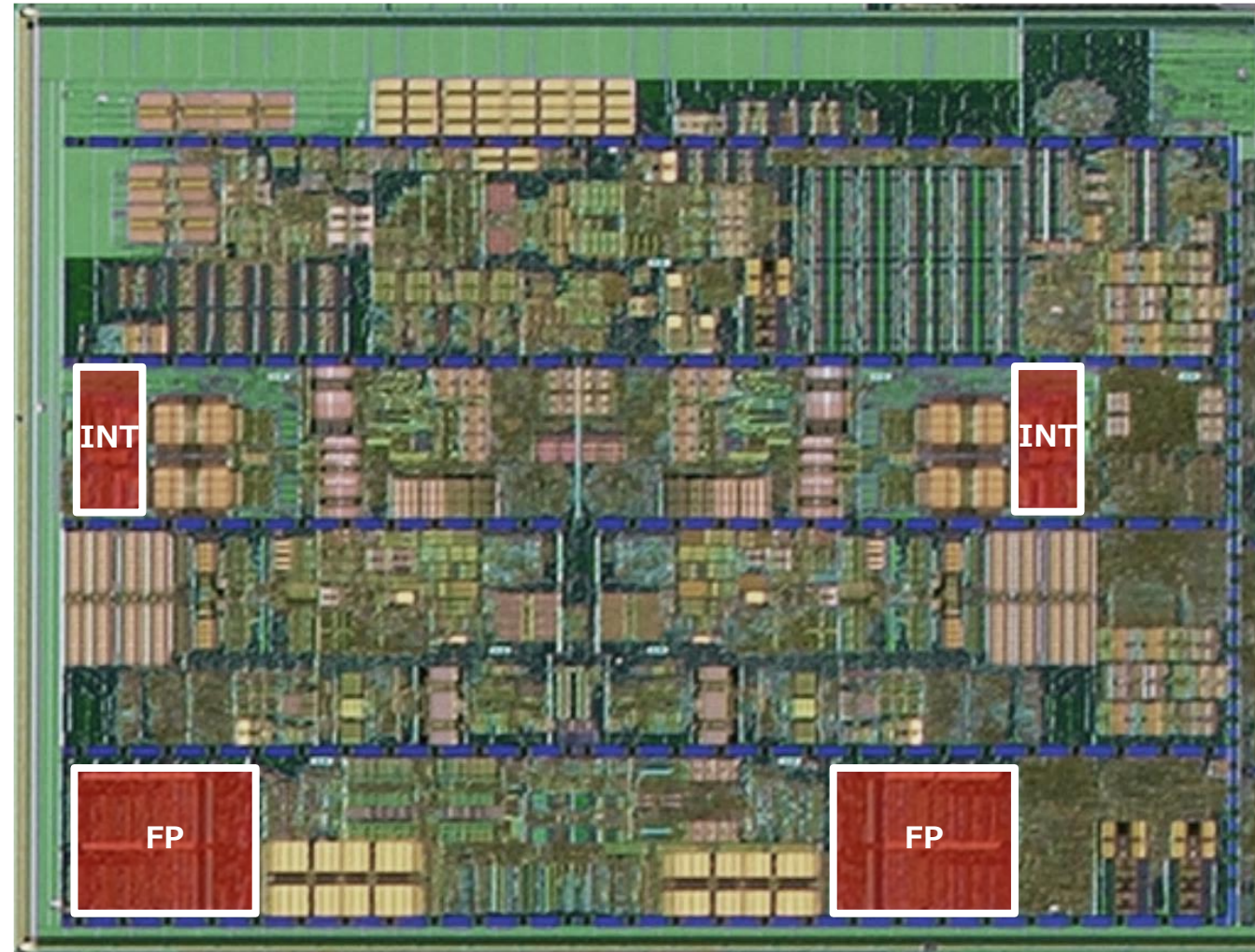
カレーのレシピ

- 1.野菜を切る
- 2.野菜を炒める
- 3.肉を切る
- 4.肉を炒める
- 5.煮込む
- 6.ルーを入れる
- 7.コメを研ぐ
- 8.コメを炊く
- 9.もりつける



AMD Bulldozer のコア部分に占める演算器部分

チップ写真は https://en.wikichip.org/wiki/File:Bulldozer_Die_Shot.jpg より



- 演算器（INT/FP）が CPU コア全体に占める割合はわずか

Apple A18/M4 CPU の場合

Cardyak's Microarchitecture Cheat Sheet より

https://docs.google.com/spreadsheets/d/18ln8SKIGRK5_6NymgdB9oLbTJCFwx0iFI-vUs6WFyuE/edit?gid=1076260513#gid=1076260513

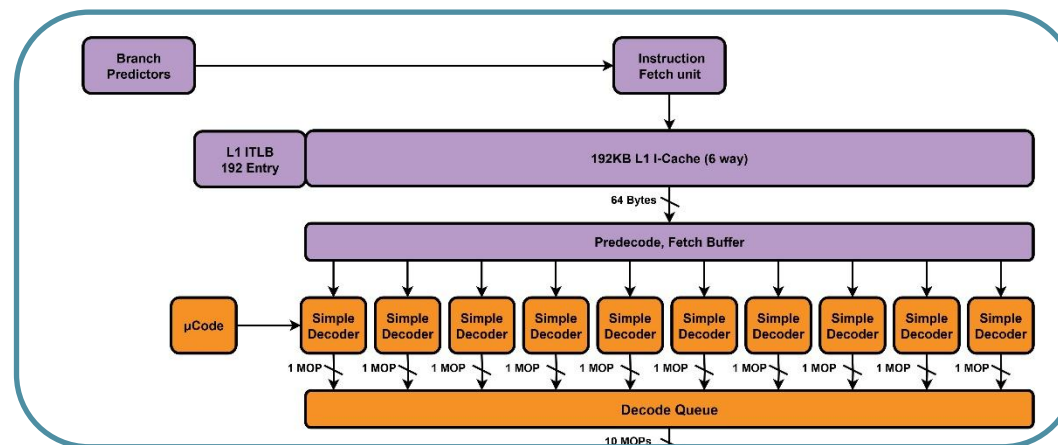
Apple - 2024

A18 P

By Cardyak

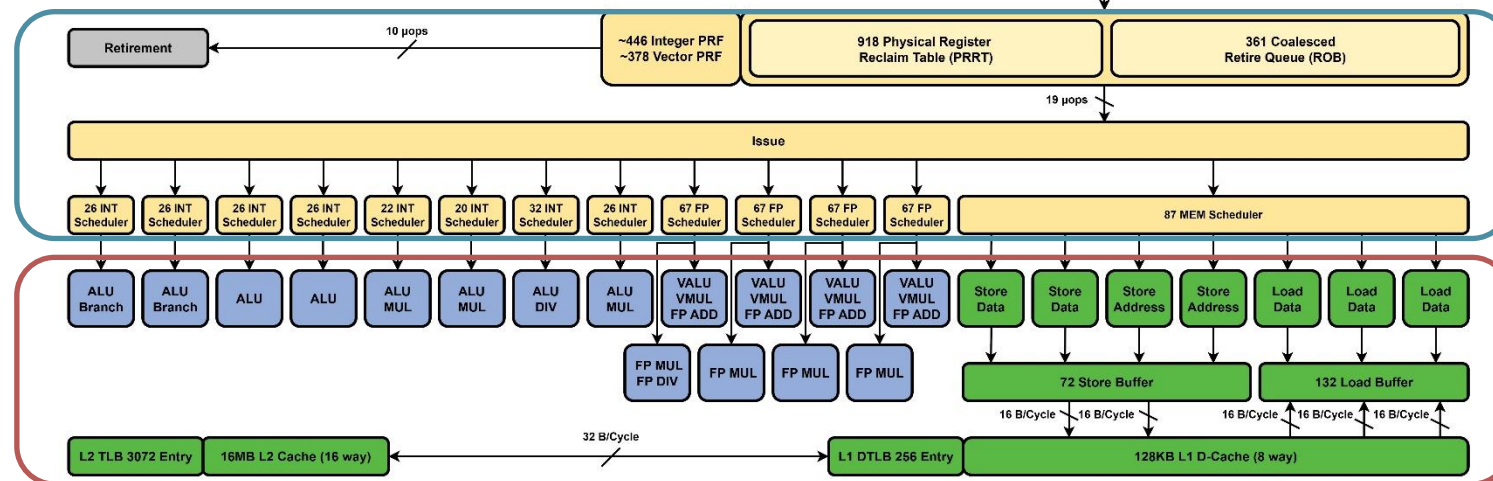
Microarchitecture Block Diagram

命令の読み出しと解釈
依存関係を解析して、
どこを並列にやっていいか
割り出す部分



依存した処理が
終わっているかを監視して、
演算器に送り出す部分

演算器



■ こういう複雑な部分（青）をなくす

NVIDIA GPU の場合

Rodrigo Huerta et al., Dissecting and Modeling the Architecture of Modern GPU Cores より <https://dl.acm.org/doi/10.1145/3725843.3756041>

- 階層型の構造をしている
 - 命令供給系（右上）
 - 4つのサブコアが上記を共有
- サブコア
 - 命令供給系
 - 16～32個の演算器で共有

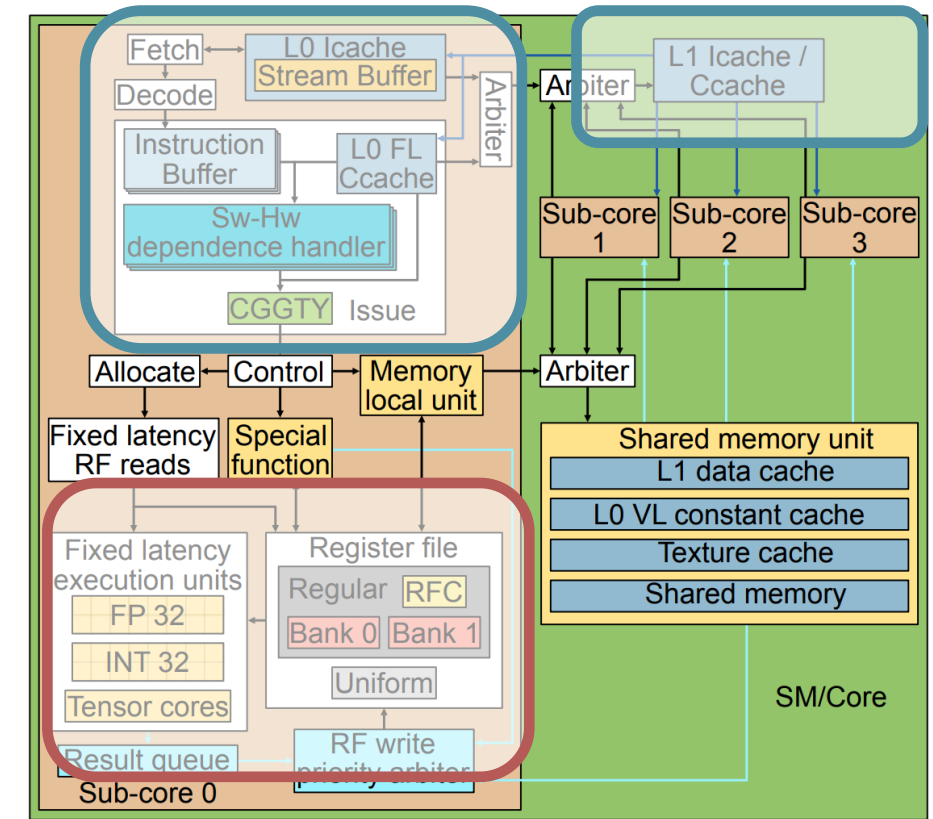
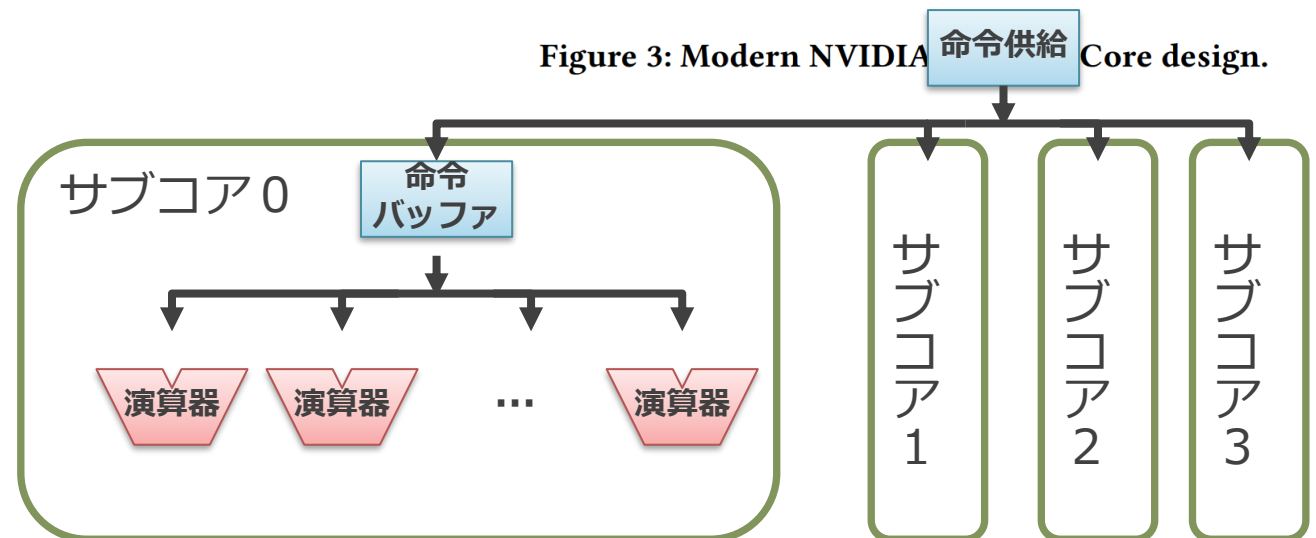


Figure 3: Modern NVIDIA Core design.



GPU における回路量当たりの性能向上

■ GPU における回路量あたりの性能向上

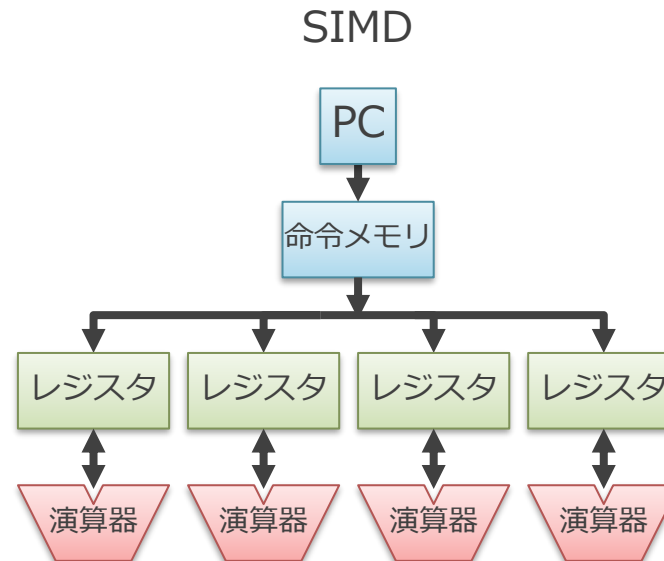
1. SIMD による制御部の共有
2. 制御部自体の小型化

■ たとえば先ほどのチップ写真の場合...

- 赤色の部分のみを N 倍に増やしても、全体は大きくは増えない
 - まず赤色自体が占める面積がかなり小さいから
- しかし非赤色の部分を減らすと、かなりの数の赤色が積める

補足：データの供給について

- ここまでは簡単のためデータの供給についての議論を無視している
 - メモリやレジスタ・ファイルは演算器間で共有できない
- ここをさらになんとかするのが、行列積に特化したハードウェア
 - TPU や Tensor Core
 - これらについては後述



もくじ

1. フリンの分類と SIMD
2. GPU における回路量当たりの性能向上
3. SIMD プロセッサのプログラミング・モデル

プログラミング・モデル

- ここまでは SIMD のハードウェアについて説明
 - ここからはその上でどのようなプログラムを走らせるのかを説明
- プログラミング・モデル
 - どのような命令によりプログラムを記述するかを規定
 - これまでに説明したハードウェアの構成方式とはある程度独立した概念
- カレー屋さんでいうと…
 - ハードウェア = お店の人員と器具をどう配置して使うか
 - プログラミング・モデル = (器具や配置を考慮した) レシピをどう表現するか

ハードウェア方式とプログラミング・モデル

■ 異なるハードウェア方式とプログラミング・モデルの組み合わせ

- NVIDIA の GPU
 - ハードウェア : SIMD (を拡張した SIMT)
 - プログラミング・モデル : Single Program Multiple Data (SPMD)
- AMD や Intel の GPU
 - ハードウェア : SIMD
 - プログラミング・モデル : 「SIMD命令」と呼ぶ形式の命令 + SPMD
- (AMD/Intel でも SPMD モデルにより書かれたプログラムをコンパイラによって SIMD 命令に変換して実行することが行われる)

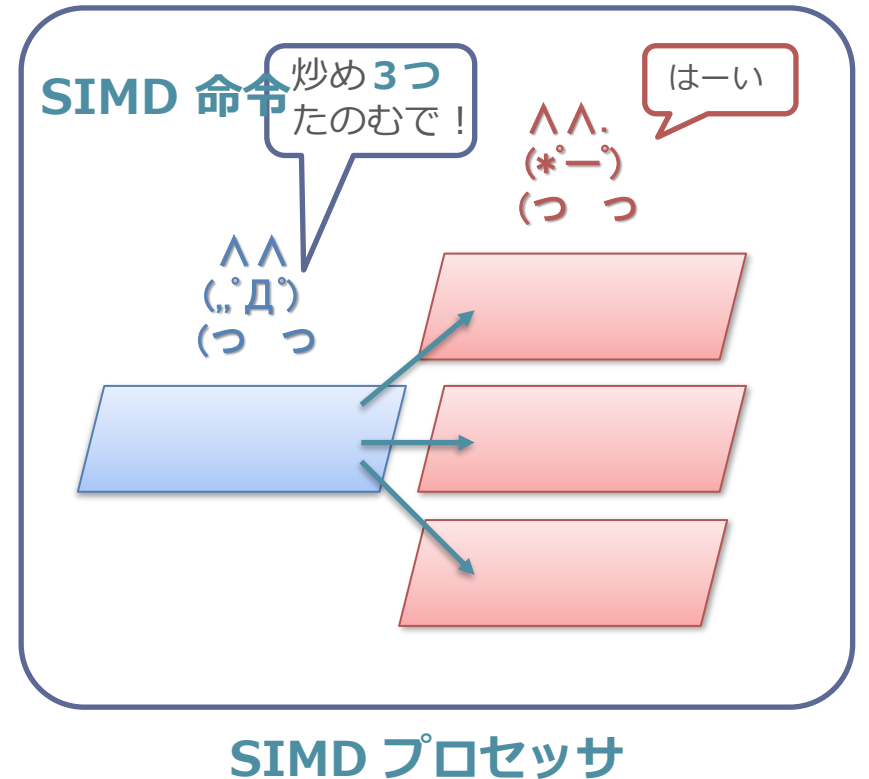
	ハードウェアの方式	プログラミング・モデル
NVIDIA (Volta)	SIMT	SPMD
AMD (GCN,RDNA)	SIMD	SPMD/SIMD 命令
Intel (GEN)	SIMD	SPMD/SIMD 命令

もくじ

1. フリンの分類と SIMD
2. GPU における回路量当たりの性能向上
3. SIMD プロセッサのプログラミング・モデル
 1. SIMD 命令方式
 2. SPMD 方式

用語の定義

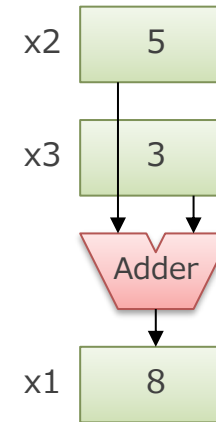
- SIMD プロセッサ :
 - 1つの命令を解釈し、複数のデータに対して同じ演算を同時に行うハードウェア
- SIMD 命令 :
 - SIMD プロセッサ上で複数のデータを対象に同時に演算を行う命令
- まぎらわしい :
 - 「SIMD (プロセッサ)」はハードウェアの構成方式だが,
 - 「SIMD 命令」は命令の表現方式を意味することに注意



SIMD 命令：複数のデータを対象に同時に演算を行う

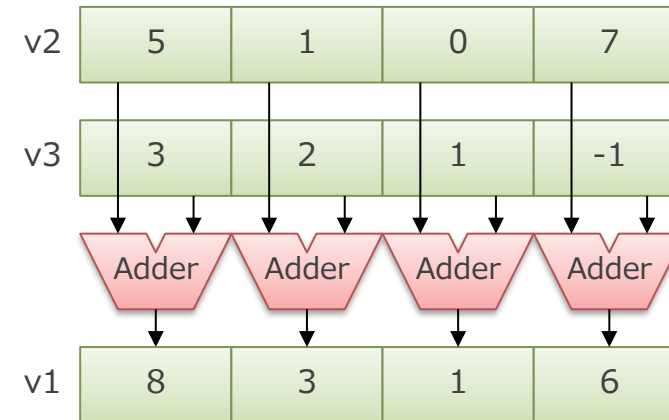
■ 通常の命令（スカラー命令）

- $\text{add } x1 \leftarrow x2 + x3$
- 「人参を 1 つ切る」



■ SIMD 命令

- $\text{add_x4 } v1 \leftarrow v2 + v3$
- $v1 \sim v3$ は 1 つのレジスタに 4 つの値が入ったレジスタ
- SIMD レジスタやベクトルレジスタと呼ばれる
- 「人参を 4 つ切る」



SIMD 命令を使う方法

■ SIMD 命令を使う方法：

1. コンパイラに頑張ってもらおう
 - 複数同時に計算しても大丈夫なところを自動で検出する
 - 後述の SPMD で書かれたプログラムならある程度できる
2. 組み込み関数を使って人間が直接書く
 - 自動でうまくいかない場合は人間が頑張る（つらい）

(余談) 組み込み関数 (intrinsic)

// 連続した加算を4つ行う SIMD 命令に相当する関数
// **コンパイル時にこの関数と等価な結果が得られる SIMD 命令に変換される**

```
void add_x4(int* dst, int* src1, int* src2) {  
    dst[0] = src1[0] + src2[0];  
    dst[1] = src1[1] + src2[1];  
    dst[2] = src1[2] + src2[2];  
    dst[3] = src1[3] + src2[3];  
}
```

// 連続した乗算を4つ行う SIMD 命令に相当する関数

```
void mul_x4(int* dst, int* src1, int* src2) {  
    dst[0] = src1[0] * src2[0];  
    dst[1] = src1[1] * src2[1];  
    dst[2] = src1[2] * src2[2];  
    dst[3] = src1[3] * src2[3];  
}
```

(余談) 組み込み関数の使用例

```
■ // src1 と src2 から n個 のベクトルの
// 内積を行い, 返り値として返す関数
int dot_product(int* s1, int* s2, int n) {
    int r = 0;
    for (int i = 0; i < n; i++) {
        r += s1[i] * s2[i];
    }
    return r;
}
```

```
■ // ベクトルの内積を行う関数の SIMD 命令版
// データの個数は4の倍数であるとする
int dot_product_x4(int* s1, int* s2, int n) {

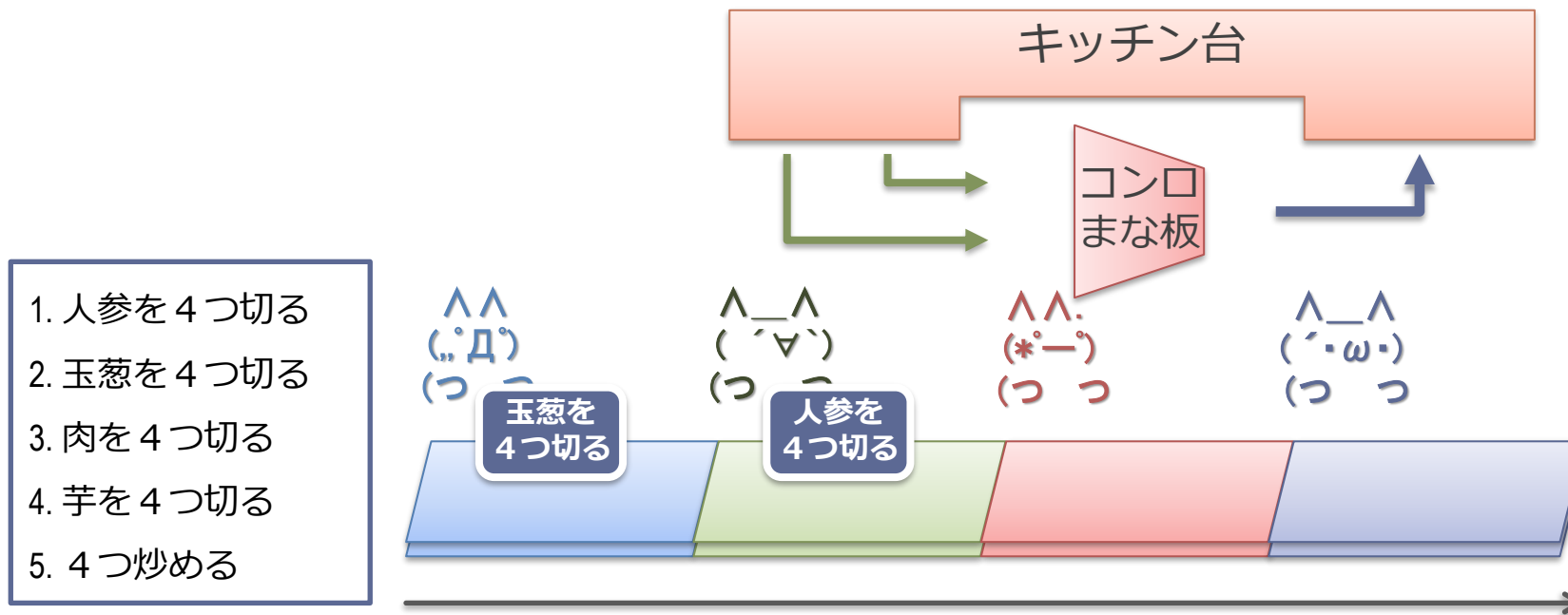
    // 4並列で内積の処理を行う
    int r_x4[4] = {0, 0, 0, 0};
    for (int i = 0; i < n; i+=4) {
        int tmp_x4[4];
        mul_x4(tmp_x4, s1 + i, s2 + i);
        add_x4(r_x4, r_x4, tmp_x4);
    }

    // 4つの中間結果をまとめる
    int r = 0;
    for (int i = 0; i < 4; i++) {
        r += r_x4[i];
    }
    return r;
}
```

■ mul_x4 や add_x4 の部分が SIMD 命令に置き換わる形でコンパイルされる

◇ これを人手でやるのは結構しんどい

カレー屋さんの場合の SIMD 命令や組み込み関数



- SIMD 命令や組み込み関数 :
 - レシピに「4つ切る」と最初から書いてある
- プログラミング（レシピを書くとき）なにが難しいのか？
 - 都合よく4つかレーの注文が来ない時の対応
 - 材料を全部4つセットで並べなて、一気に取れるようにしないといけない

2つのプログラミング・モデル

1. SIMD 命令方式
2. SPMD方式

SIMD 命令と SPMD

■ SIMD 命令

- SIMD のハードウェアを直接命令に見せている
- 命令の形：「人参を 4 つ切って」

■ Single Program Multiple Data (SPMD) :

- マルチスレッド方式によってプログラムを記述
 - Single Program = 同じことをするスレッド
 - 冒頭の行列積の例そのもの
- SIMD のハードウェアはプログラマからは直接見えないよう隠蔽される
- 命令の形：「人参を 1 つ切って」
(具体的にいくつ切られるかはハードで決まる)

SPMD による並列化の例

- ループで書いた行列積のコード

```
for (k = 0; k < 1024; k++)  
  for (j = 0; j < 1024; j++)  
    for (i = 0; i < 1024; i++)  
      a[j][i] += b[j][k] * c[k][i];
```

- j の部分のループを複数のスレッドとしてマルチスレッド化

// func が 1024 個起動されて並列に実行される

// 0-1023 がスレッド ID として渡される

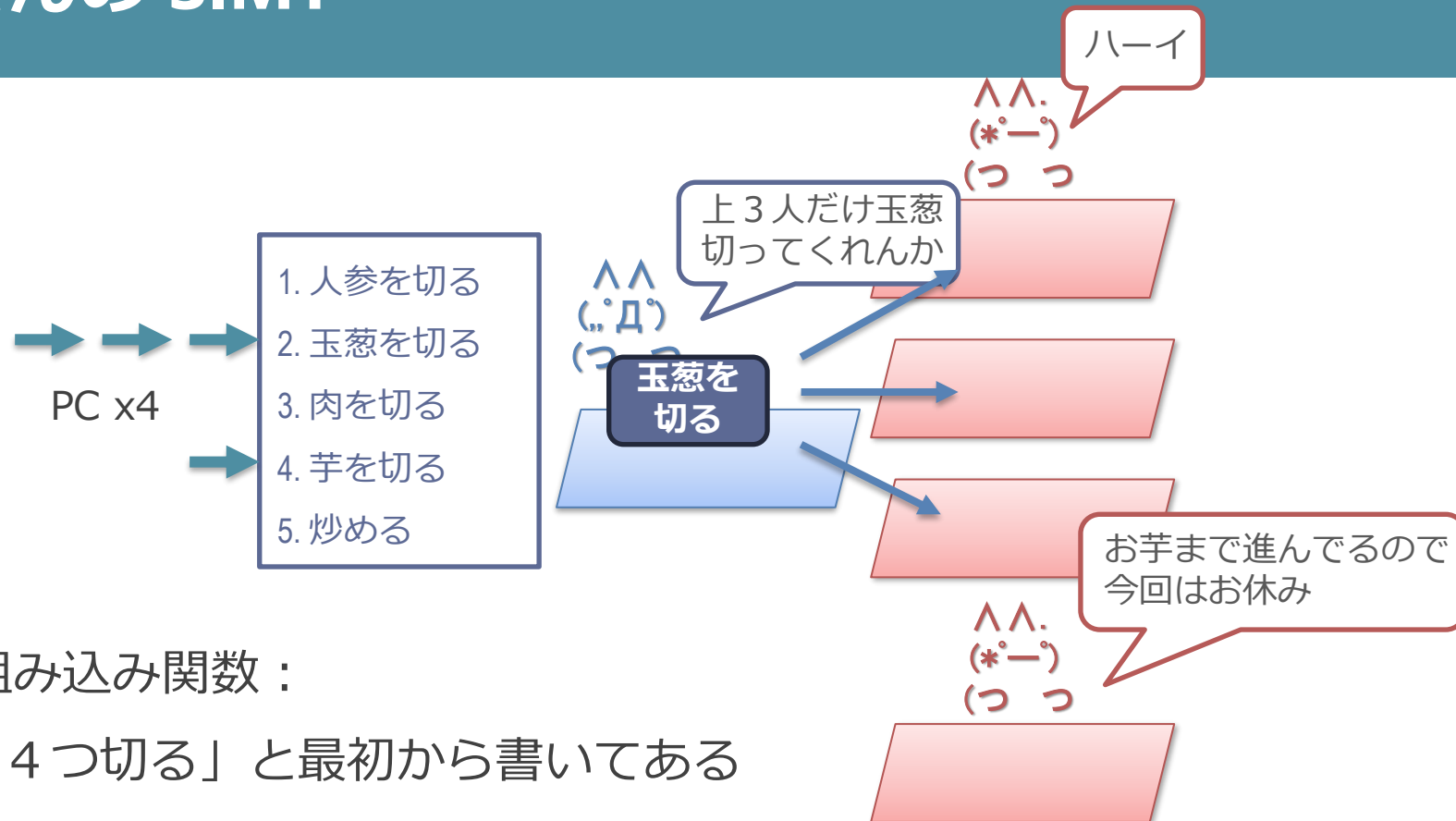
```
func(thread_id) {  
  j = thread_id; // k だと同じ場所を書いてしまう  
  for (k = 0; k < 1024; k++)  
    for (i = 0; i < 1024; i++)  
      a[j][i] += b[j][k] * c[k][i];  
}
```

SPMD で書かれたプログラムの実行

■ 以下の 2 通りの方法がある

1. SIMD 命令への変換
 - コンパイラにより自動で変換
 - 変換された SIMD 命令を実行
 - AMD/Intel の方式
2. Single Instruction Multiple Thread (SIMT) ハードウェアによる実行
 - 命令列自体はスカラー命令で記述
 - マルチスレッド実行をハードウェアで SIMD にマップする
 - NVIDIA の方式

カレー屋さんの SIMT



■ SIMD 命令や組み込み関数 :

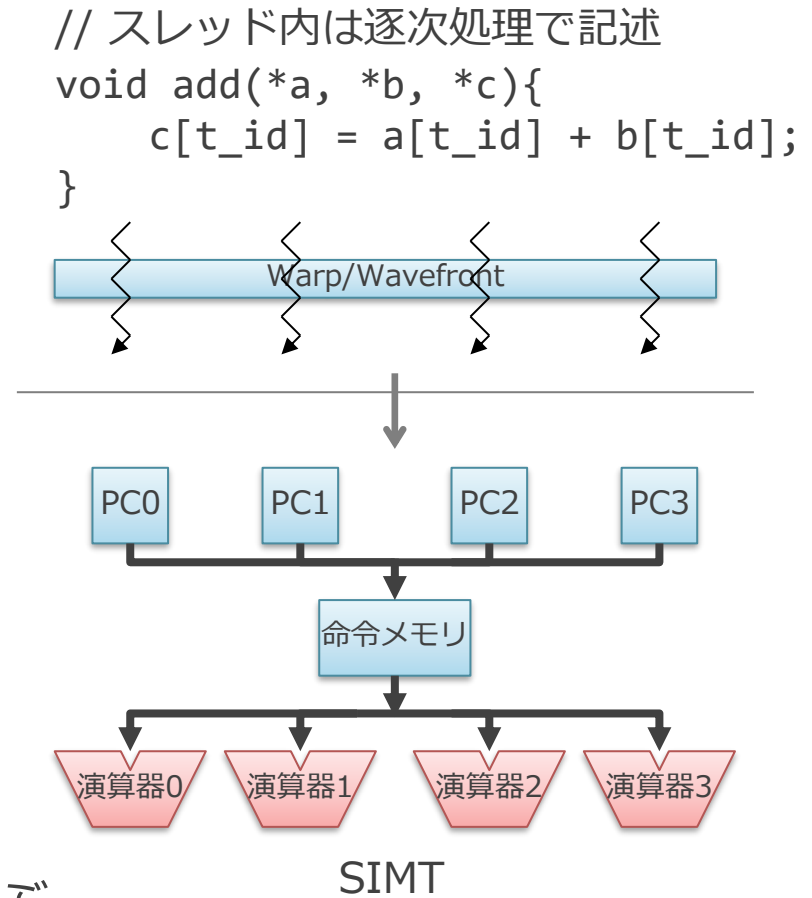
- レシピに「4つ切る」と最初から書いてある

■ SIMT :

- レシピ自体は「1つ切る」とだけ書いてある
- かわりに PC を4つ持ち、都度いくつ同時にできるか考える
- PC が同じところにある人達の分は、一括して出来る

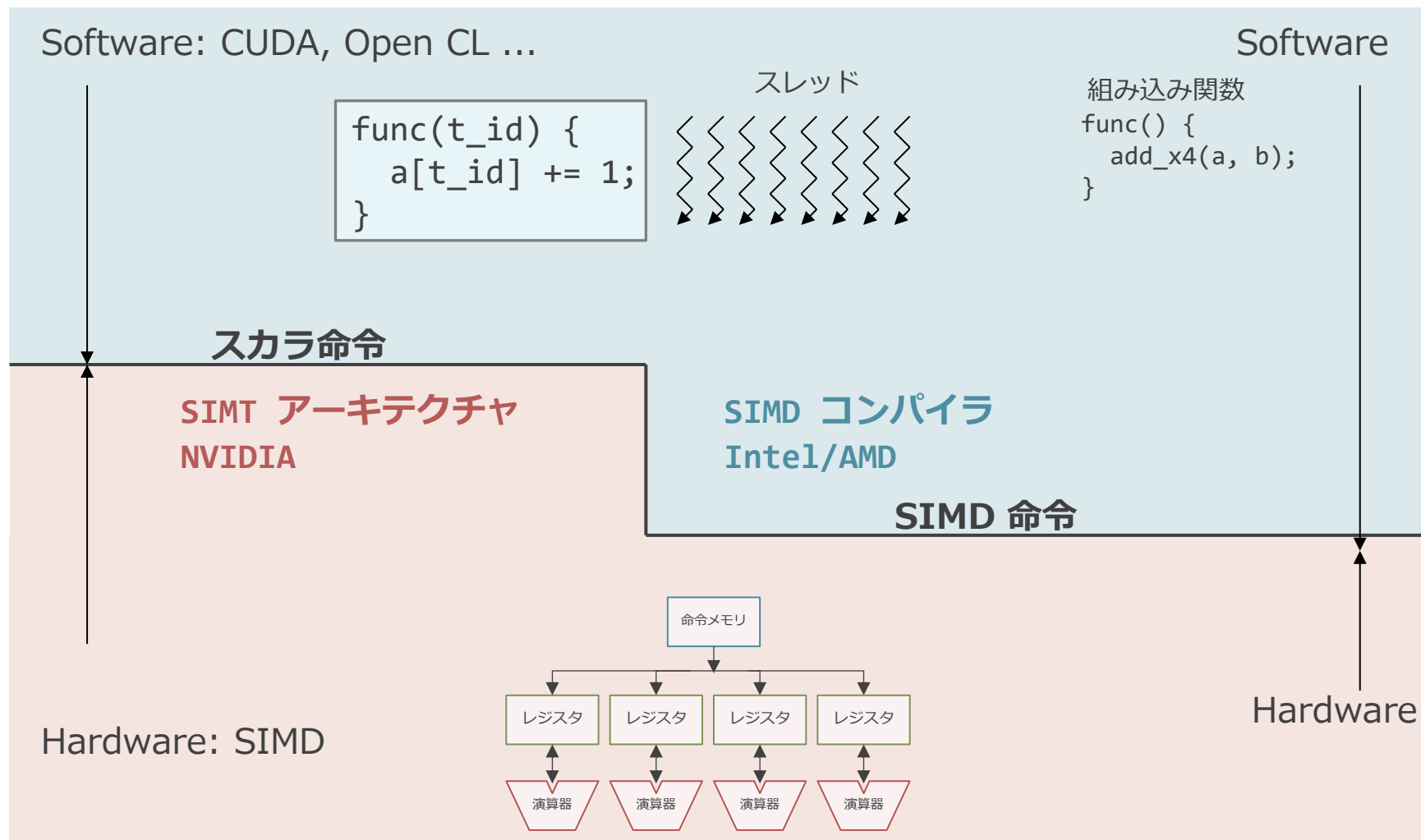
SIMT の構造

- PC を演算器の分だけ持つ
 - 命令メモリは1つのまま
- SPMD の各スレッドを PC に割り当てる
 - 基本的にはスレッドを時分割でそれぞれ実行
- ポイント：
 - 全スレッドの PC が同じアドレスを指している場合は SIMD プロセッサとして振る舞う
 - このまとまりを Warp or Wavefront と呼ぶ
 - 分岐しなければこの状態が保たれる

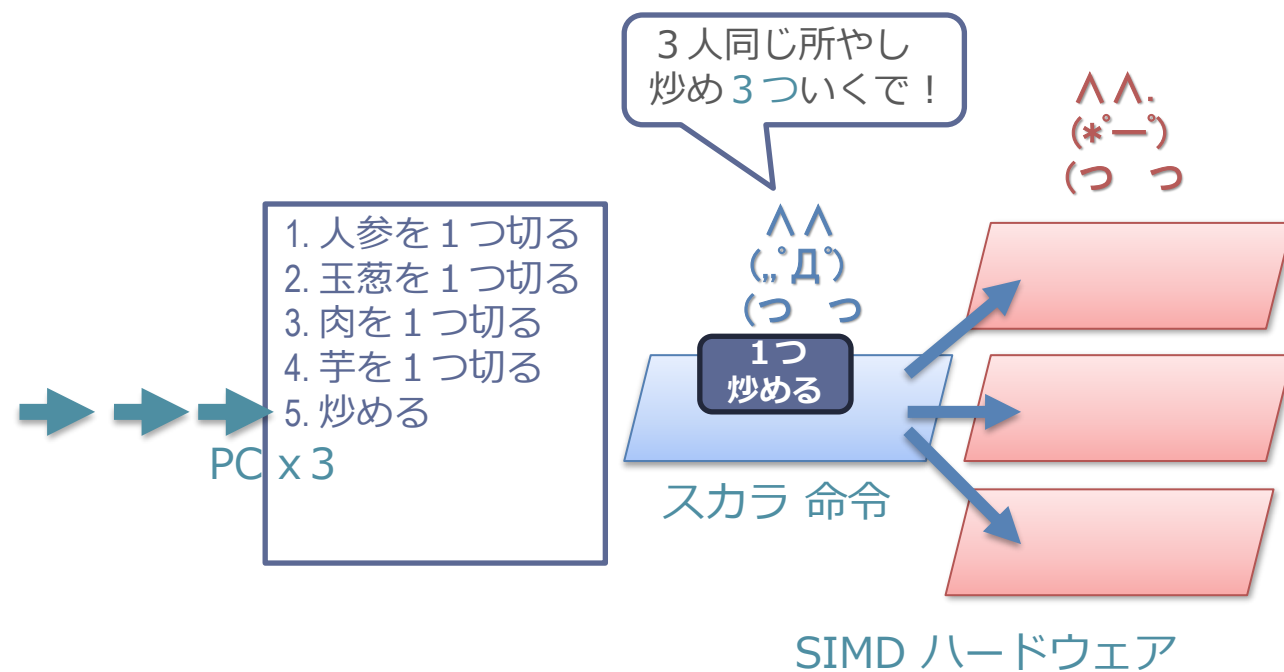
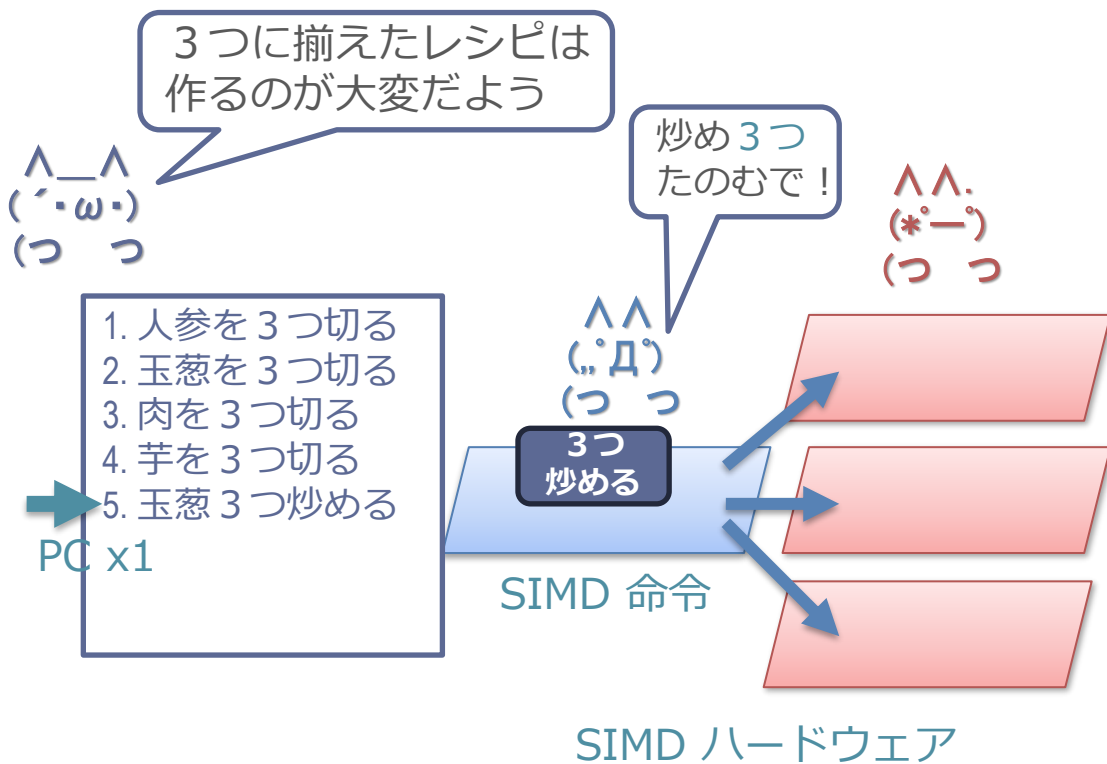


GPU におけるソフトとハードの関係

図は Caroline Collange “GPU architecture part 2: SIMT control flow management” より



カレー屋さんにおける SIMD 命令と SIMT



同じところを処理している場合に、指示をブロードキャストするのが SIMT.
ハードの形としては SIMD ハードウェアであり、この方式のことを SIMT と言う。

SIMT の利点と問題点

■ 利点：

- マルチスレッドで書かれたプログラムが SIMD プロセッサでそのまま実行できる
- = 難しいコンパイラがいらない
 - CPU 命令セットである RISC-V ベースの SIMT プロセッサも提案されている

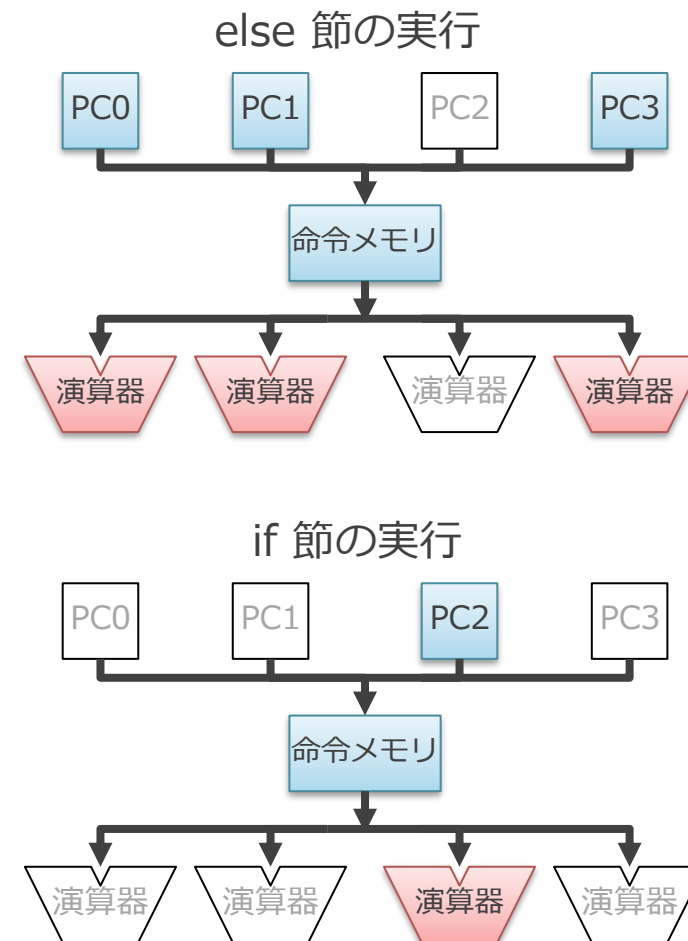
■ 問題点：

- スレッド間で違う方向に分岐すると実行速度が落ちる
 - branch divergence と呼ぶ
- 水平方向の演算が自然には表現できない

問題 1 : スレッド間で違う方向に分岐すると実行速度が落ちる (Branch divergence)

```
int branch_func(int t_id, int i) {  
    if (t_id == 2)  
        i++;  
    else  
        i--;  
    return i;  
}
```

- else と if の 2 回に分けて実行される
 - この例では実行速度が $\frac{1}{2}$ に

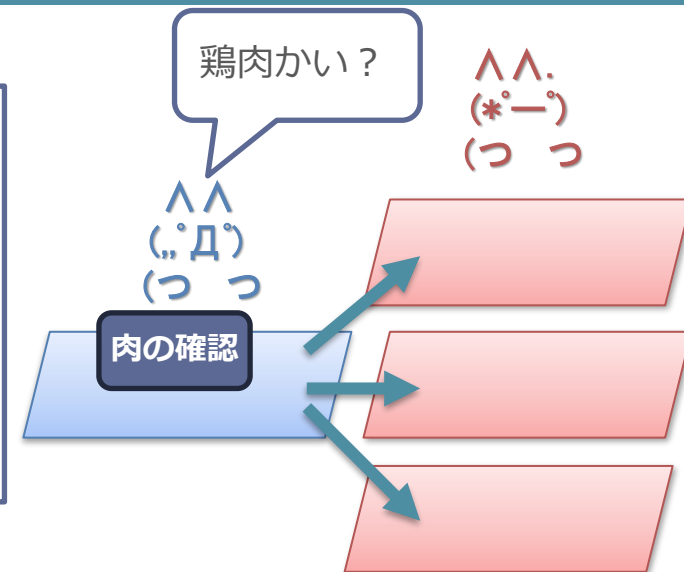


カレー屋さんにおける branch divergence

- レシピに分岐があるとする
 - 鶏肉か？牛肉か？
- レシピの「今ここやってます」が分散する
 - レシピ読み上げ指示の一人は一箇所しか読めない
- 鶏肉の処理をやってから、牛肉の処理を時分割でやる
 - 鶏肉の処理をしている間は下の2本は何もせず無駄

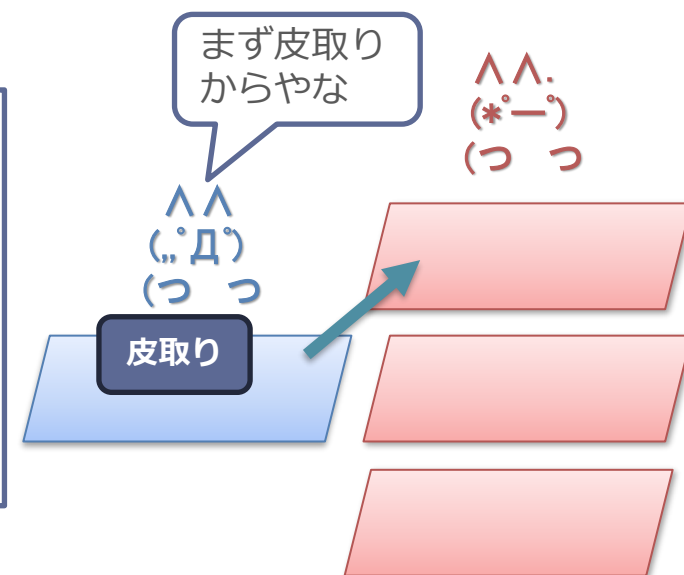


1. 人参を切る
2. もし鶏肉なら？
3. 皮を取る
4. 牛肉なら
5. 筋を切る



PC が分散

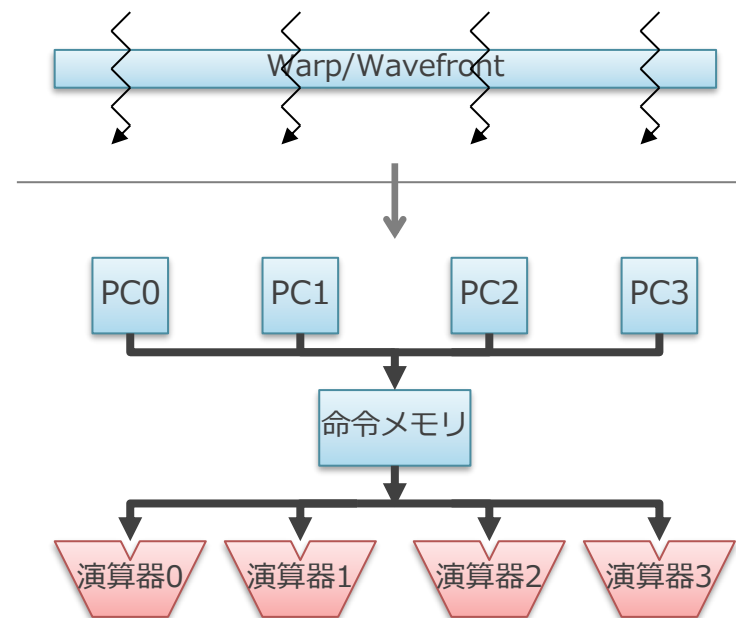
1. 人参を切る
2. もし鶏肉なら？
3. 皮を取る
4. 牛肉なら
5. 筋を切る



問題 2 水平方向の演算が自然には表現できない

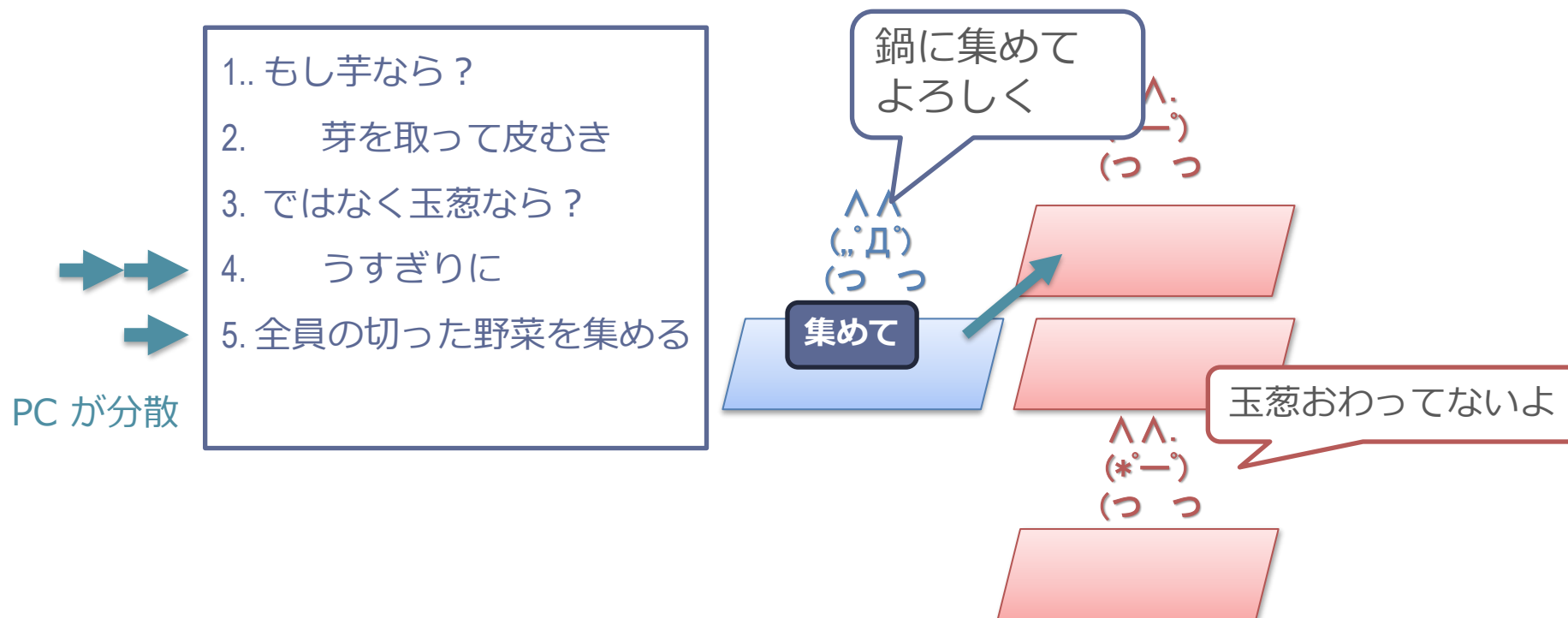
- 各演算器は独立したスレッドに割り当てられる
- 横にある演算器間では直接データのやり取りができない
 - 横で何が行われているかは保証できないため、そのようなプログラムが記述できない
 - SIMD 命令であれば、そのようなプログラムが書ける

```
// スレッド内は逐次処理で記述  
void add(*a, *b, *c){  
    c[t_id] = a[t_id] + b[t_id];  
}
```



カレー屋さんにおける 水平演算

- 「全員の野菜を集めて鍋にいれる」みたいな操作が難しい
 - 全員がそこに揃ってる事が保証できないため
 - 分岐があると時分割で処理される
 - 玉葱の薄切りが終わってないかもしれない
 - 「ここで全員を待つ」みたいな指示が必要



SIMD 命令と SIMT のまとめ

- SIMD プロセッサをどのように使うか
 - SIMD 命令モデル vs. SIMT モデル
 - (モデルの名前が適切ではない気もする)
 - 異なる層で、複数演算器の同時処理の対応を行っている
 - SIMD 命令： コンパイラ or 人手
 - SIMT： ハードウェア

GPU のアーキテクチャのまとめ

- 下記の性質を利用

1. 並列に動作する大量のスレッドがある
2. 各スレッドは基本的に同じことをしている

- 演算器以外の回路を削減

- 制御部を共有してハード量当たりの性能を向上
- 制御部自体の小型化

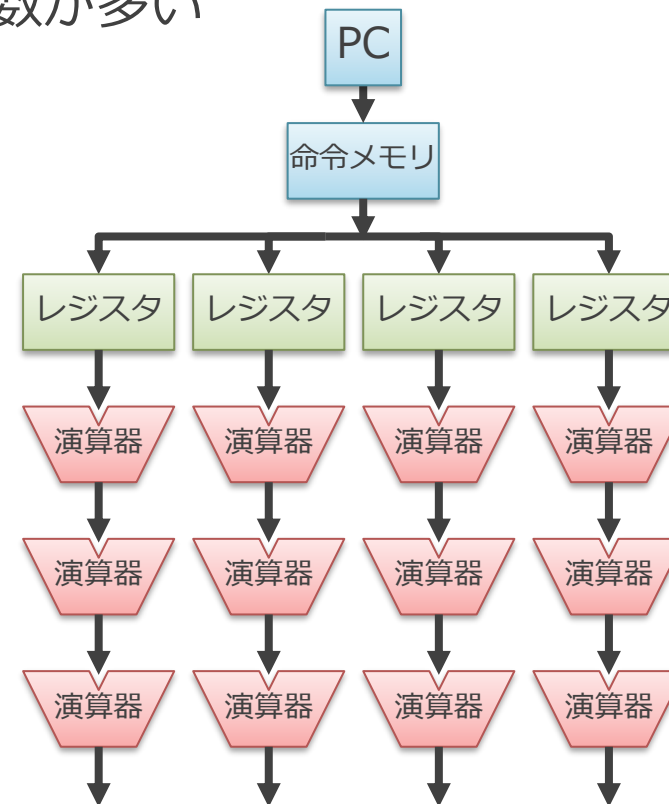
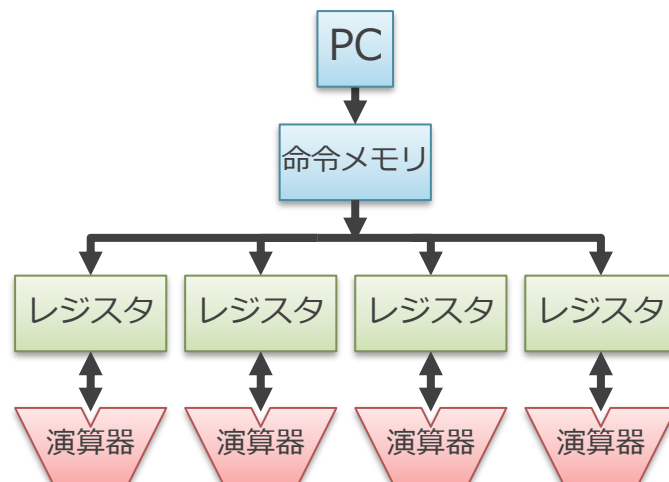
アクセラレータについて

アクセラレータや GPU は何故 CPU より速いのか？

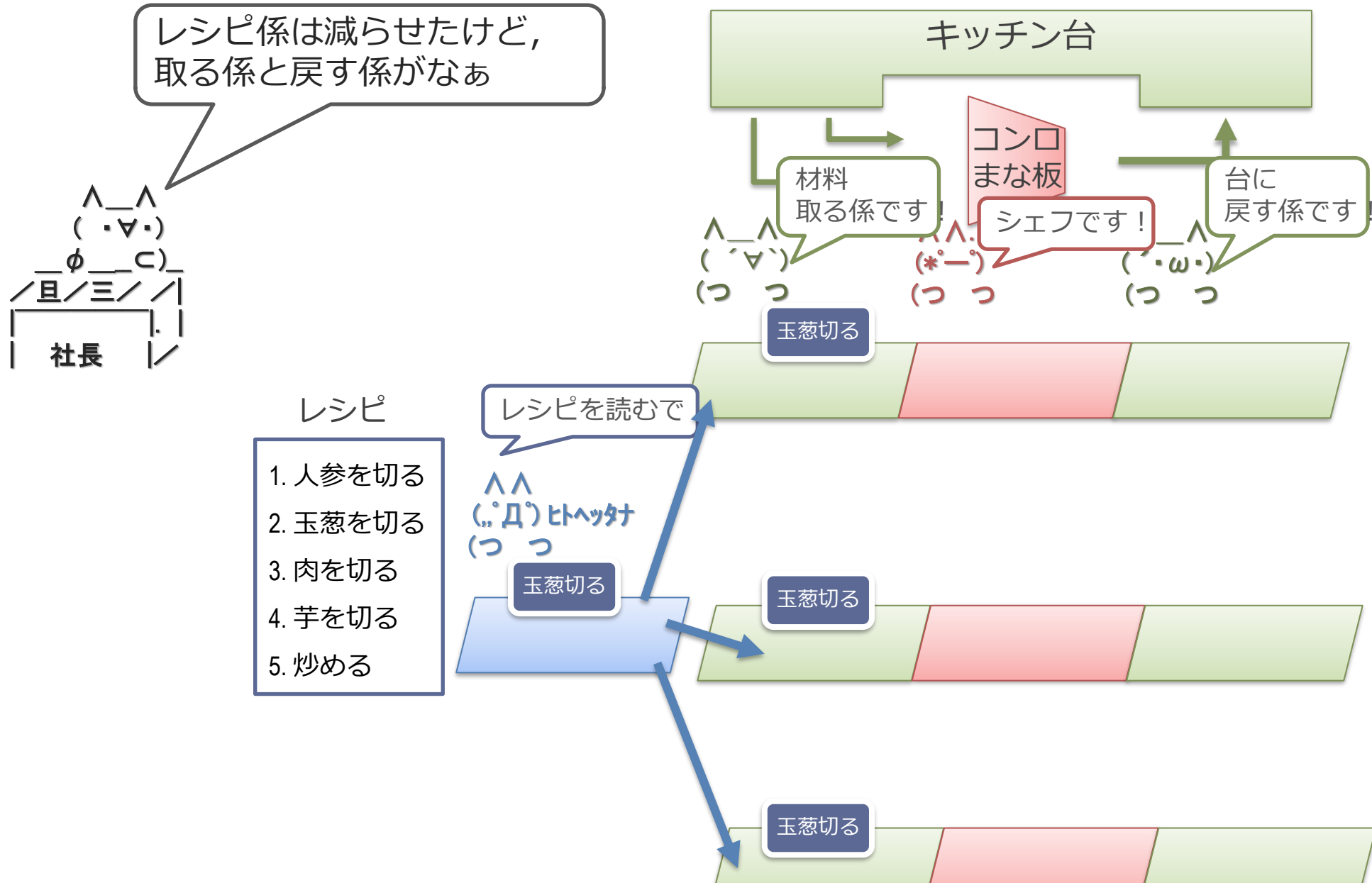
- アクセラレータ
 - 特定の問題に特化したハードウェア
- 演算器そのものは一緒
 - 個々の加算器や乗算器などはどのアーキテクチャでも共通
- 制御部やデータのやりとりをいかに減らすかにより決まる
 - 対象のプログラムの性質に特化することで、機能を削っても性能が落ちないようにする
 - 減らした分だけよりたくさん演算器がつめる

行列積アクセラレータの場合

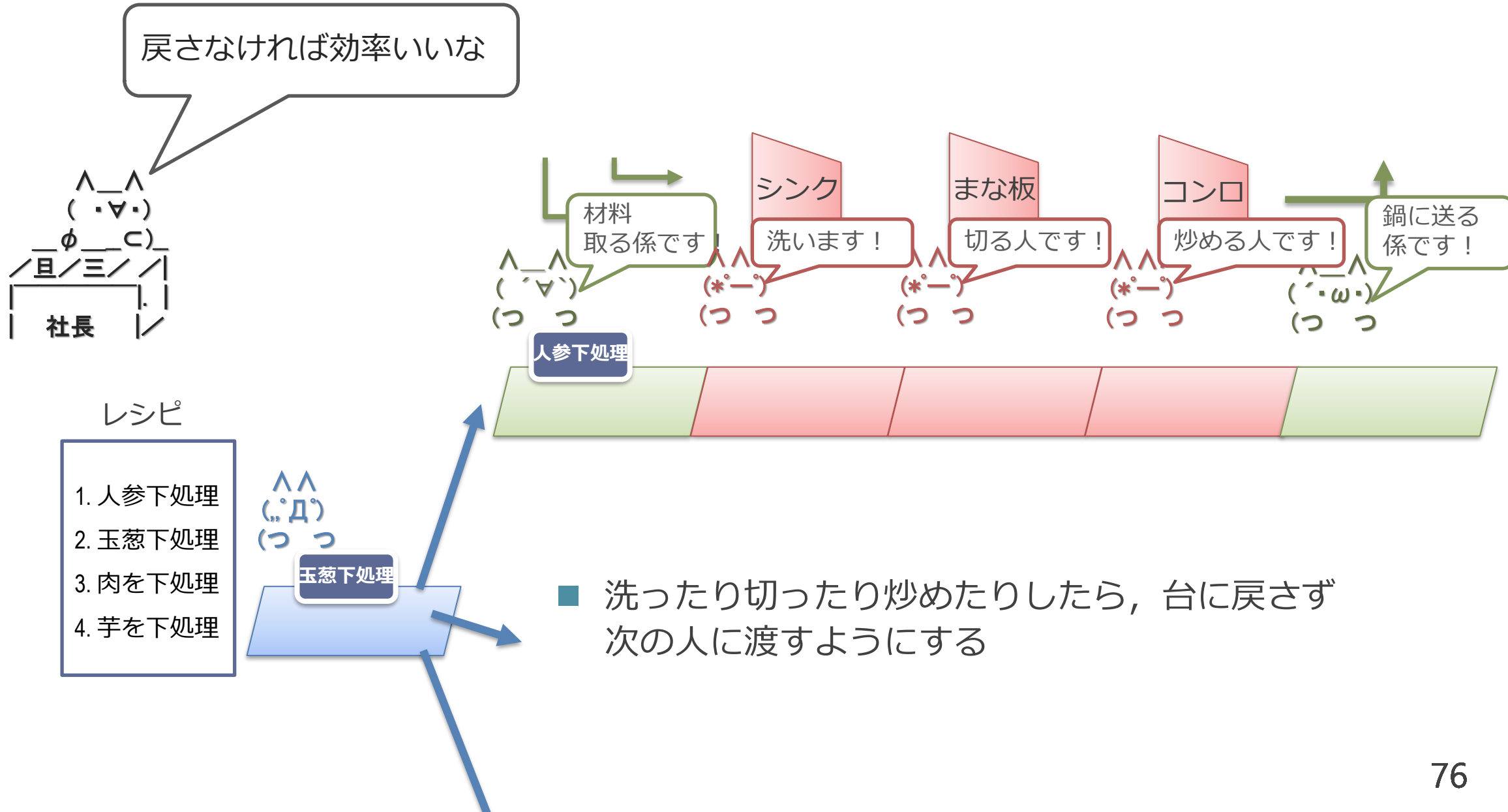
- 行列積では、単純な GPU はまだ無駄がある
 - 演算1回ごとに命令を読んだり、レジスタ・ファイルにアクセス
- 演算器を直接繋いでそこにデータを流せば良い
 - 行列積は入力データ数と比べて演算数が多い
 - $O(N^2)$ vs. $O(N^3)$
 - Google TPU や NVIDIA Tensor Core



カレー屋さんの場合：キッチン台に毎回戻るのが無駄



カレー屋さんの場合：戻さず流れ作業に

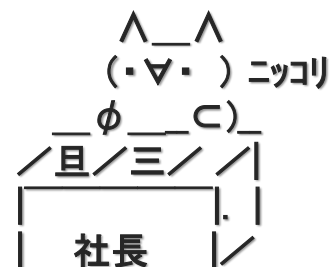


カレー屋さんの場合

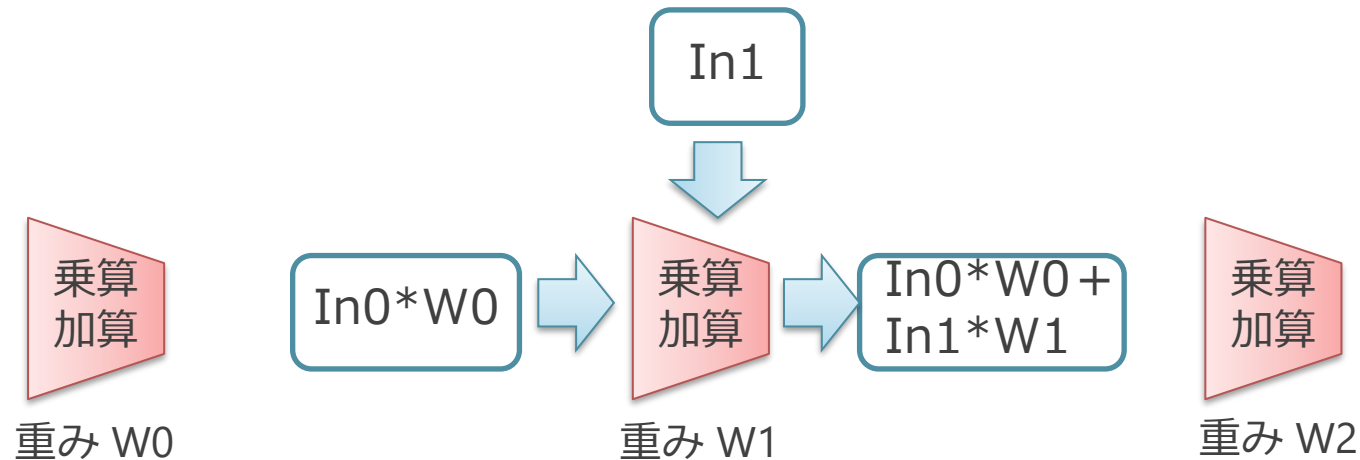
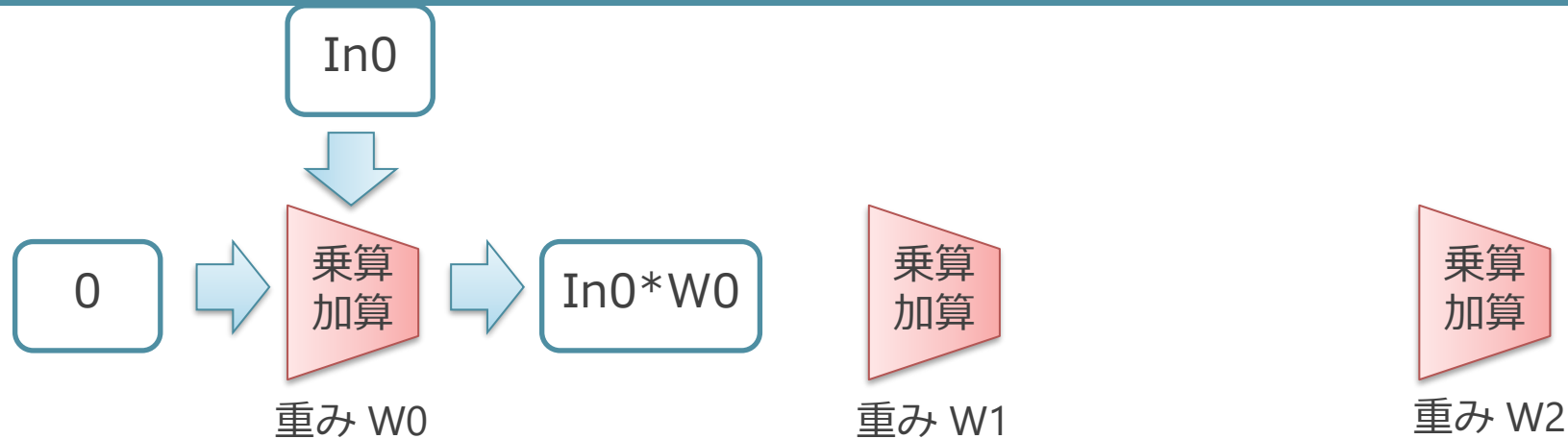


- 全体の数はず変わらず，赤い部分（演算器）の数が増えている

- Before: 洗う or 切る or 炒める，が合計 5 つできる
- After: 洗う and 切る and 炒める，が 3 セットできる
 - 炒め終わったものが 3 つ毎回できる
 - 処理としては合計 9 つできている

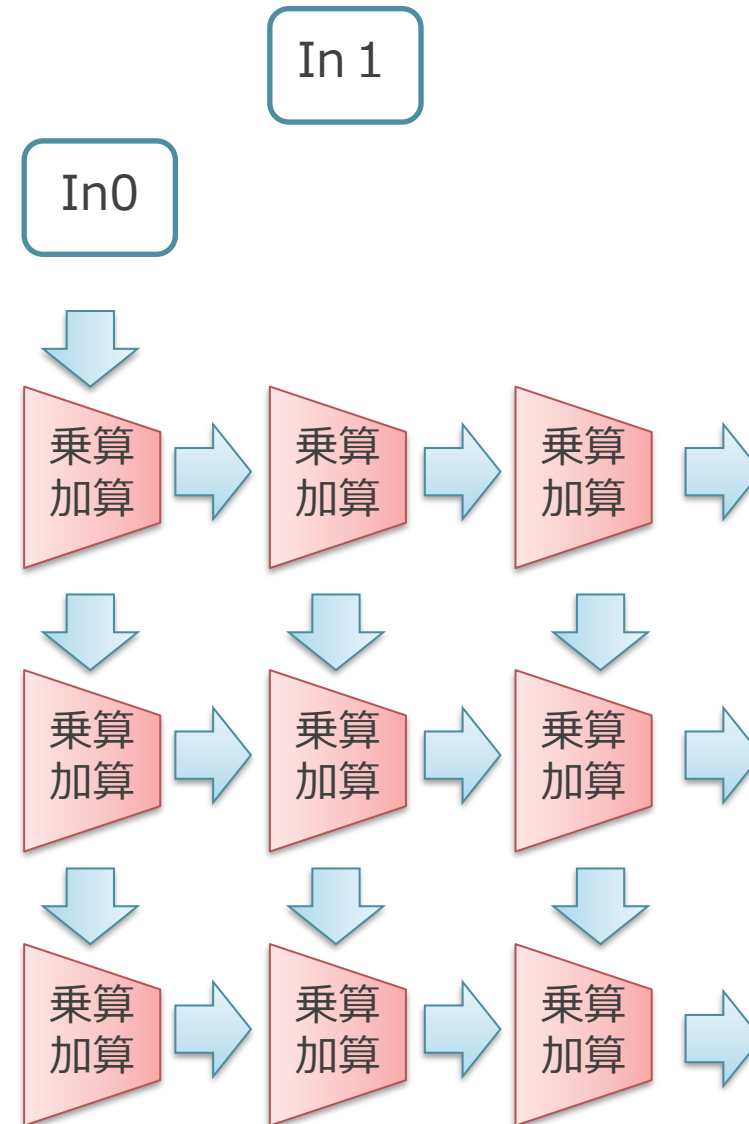


行列積の場合：シストリックアレイ



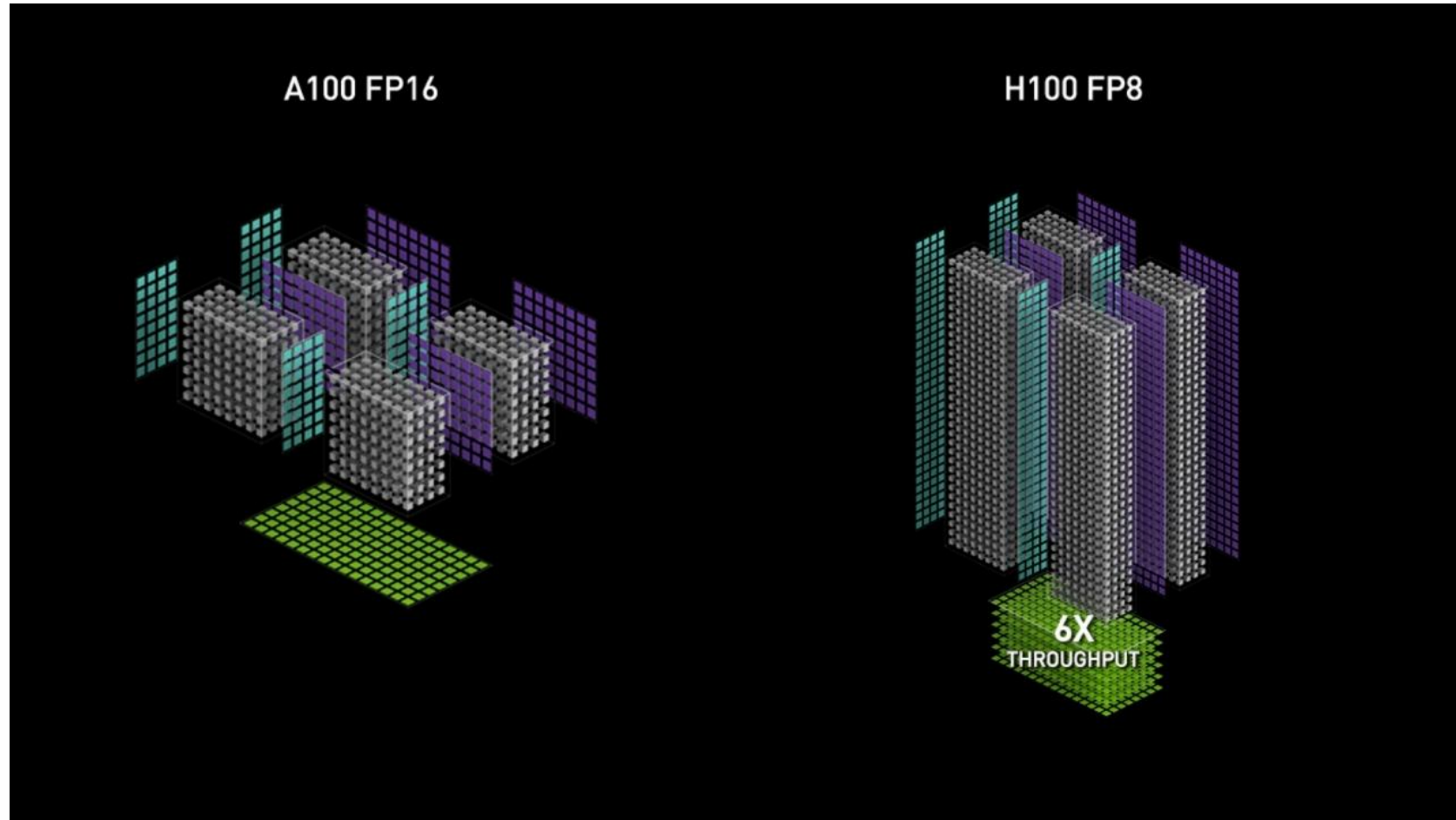
- 上から来た Input に Weight をかけて、左から来たものに足し、右に流す
- 右端からドット積の結果がでてくる

- 2次元的に同様にデータを流していく
 - 上からいれるデータは, 流れてくるものとタイミングを合わせてずらす
 - $In0*W0 + In1*W1$
- シストリック：心臓の脈打ち
 - ドクン ドクンとデータが処理されて流れていく
- Input/Output/Weight のどれを演算器に貼り付けるかで, いくつかやり方がある



Tensor Core (NVIDIA Ampere と Hopper のもの)

<https://www.nvidia.com/ja-jp/data-center/tensor-cores/> より



- NVIDIA GPU の Tensor Core は、より直接に行列積を一気に行う

Google TPU v1 の構造

Norman P. Jouppi et al., In-Datcenter Performance Analysis of a Tensor Processing Unit (ISCA 2017)より

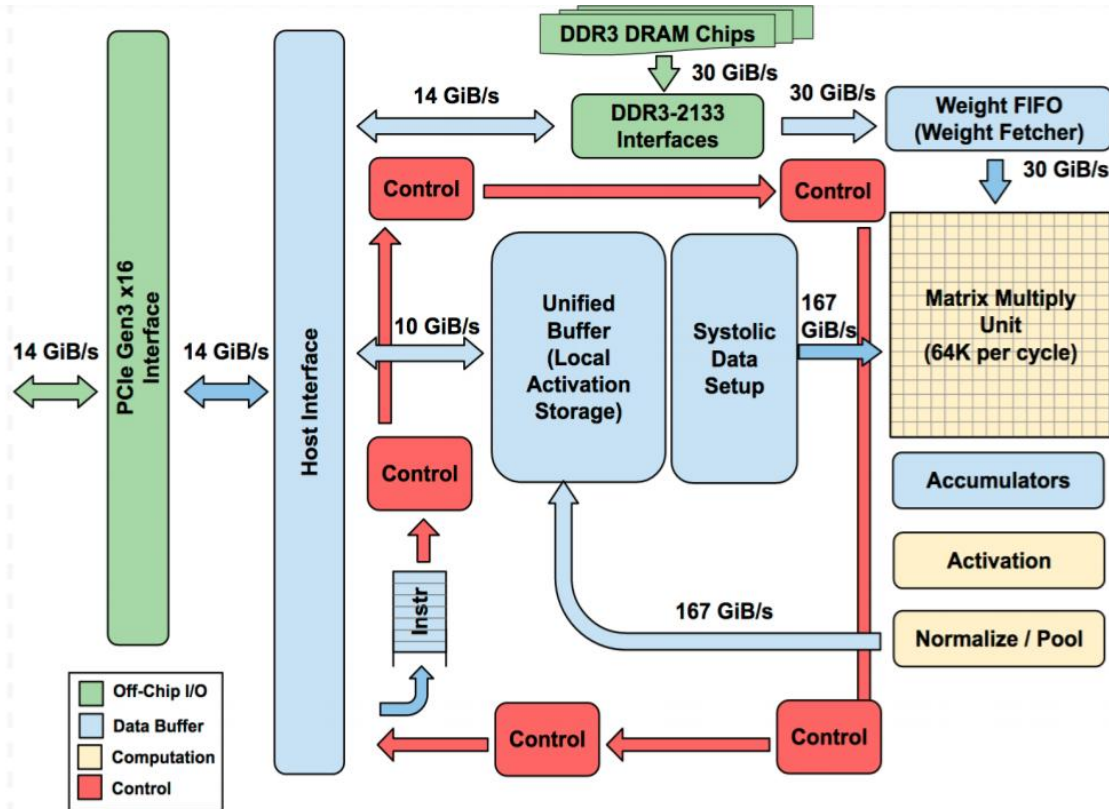


Figure 1. TPU Block Diagram. The main computation is the yellow Matrix Multiply unit. Its inputs are the blue Weight FIFO and the blue Unified Buffer and its output is the blue Accumulators. The yellow Activation Unit performs the nonlinear functions on the Accumulators, which go to the Unified Buffer.

■ 構成：

- 行列積を行うシストリックアレイ
- 活性化関数进行处理する SIMD

■ この構成はよくある

META (旧 Facebook) MTIA v2 の構造

Joel Coburn et al. Meta's Second Generation AI Chip: Model-Chip Co-Design and Productionization Experiences (ISCA 2025) より

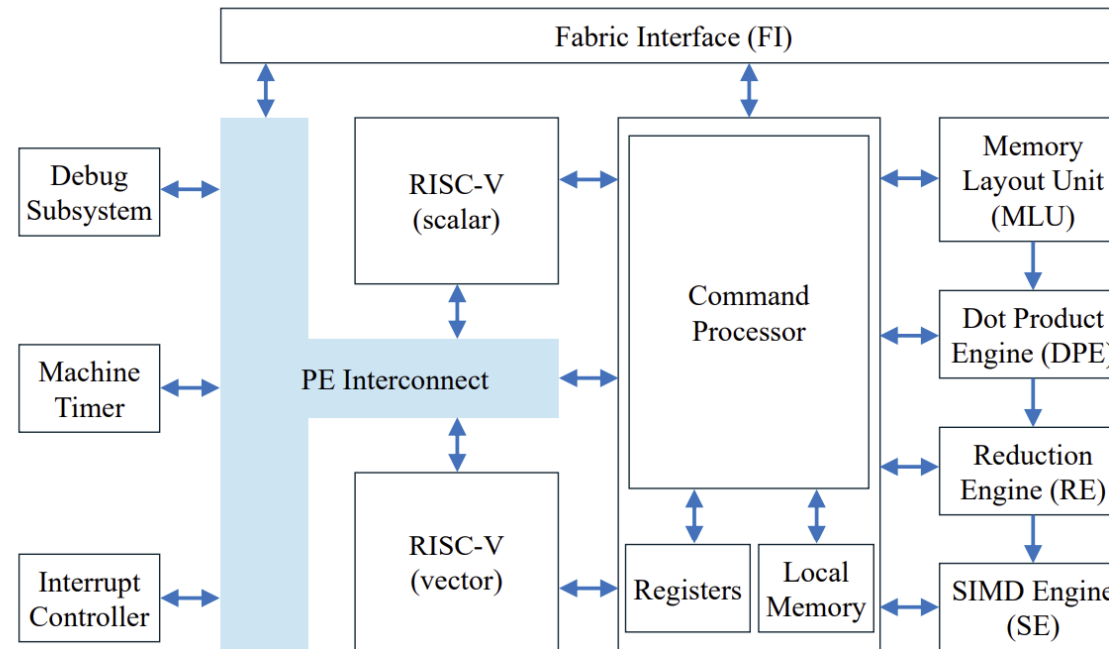


Figure 2: Internal architecture of Processing Element (PE).

■ 先ほどと同様の構造

- Dot Product Engine が行列積
- Reduction Engine と SIMD Engine が活性化関数処理

■ AI フレームワーク対応のために、柔軟性がある RISC-V CPU がのっている

アクセラレータのまとめ

- 行列積では、演算器を並べて効率を向上
 - 命令の制御だけでなく、レジスタの読み書きも省略できる
- AI アクセラレータの場合、以下の構成が典型的
 - 行列積エンジン
 - 活性化関数処理する SIMD エンジン
 - 活性化関数の処理は、あまり数珠つなぎにできないため、SIMD になっていることが多い

まとめ

まとめ：カレー屋さんと GPU

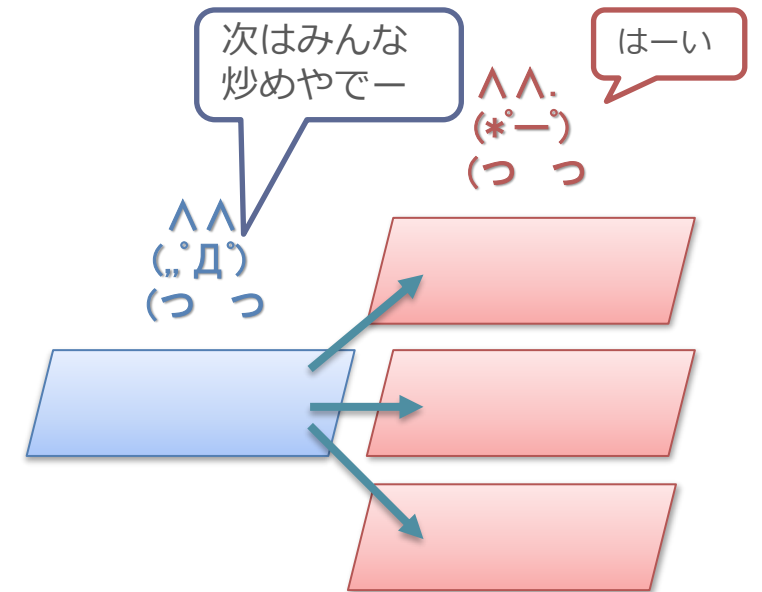
■ 前提：

- 1つのカレーを作るのではなく、大量の同じカレーを作るのが目的
- レシピを読む人やキッチン台担当の人を減らしても、同じカレーなら大量に作れる

■ 効率の上げ方：

- レシピを読む人の数を減らし、シェフ人数を増やす
- レシピを読む人に難しいことをさせず、人件費を下げる
 - 難しい事 = レシピの中で並列に出来ることを探す

■ 根っこの部分では GPU は同じことをしている



まとめ : GPU と CPU の違い

- GPU は CPU とは何が違うのか？
 - 制御部を共有して回路全体に占める演算器比率をあげている (SIMD)
 - 制御部で難しいことをさせず、小型化しその分演算器比率を上げる
- なぜ GPU やアクセラレータは、AI で高速で効率が良いのか？
 - 行列積や活性化関数は、同じ処理を複数のデータに対して処理している
 - 演算器アレイや SIMD 型の構造で、余計なことをせずに効率よく処理できる

補足：GPU についての他所の講義資料について

- Caroline Collange (フランス国立情報学自動制御研究所),
"GPU architecture",
<https://team.inria.fr/pacap/members/collange/>
- Juan Gómez Luna and Onur Mutlu (チューリッヒ工科大学),
"Computer Architecture Lecture 7: SIMD Processors and GPUs",
<https://safari.ethz.ch/architecture/fall2018/lib/exe/fetch.php?media=com-parch-fall2018-lecture7-simdandgpu-afterlecture.pdf>

より GPU の事を勉強したい人はたとえば上記の講義資料がおすすめです

補足：本日触れなかった GPU に関する話題

- マルチスレッド実行によるレイテンシの隠蔽
 - GPU では非常に多くのスレッドを同時に実行することで,
 - メモリアクセスを並列させることでレイテンシを隠蔽している
 - バックエッジに対する対処を行っている
 - SIMT や SPMD においてスレッドが並列に実行されているのとは直交した概念
- 演算精度の低精度化
 - 少しだけ補足

バックエッジに対する対処

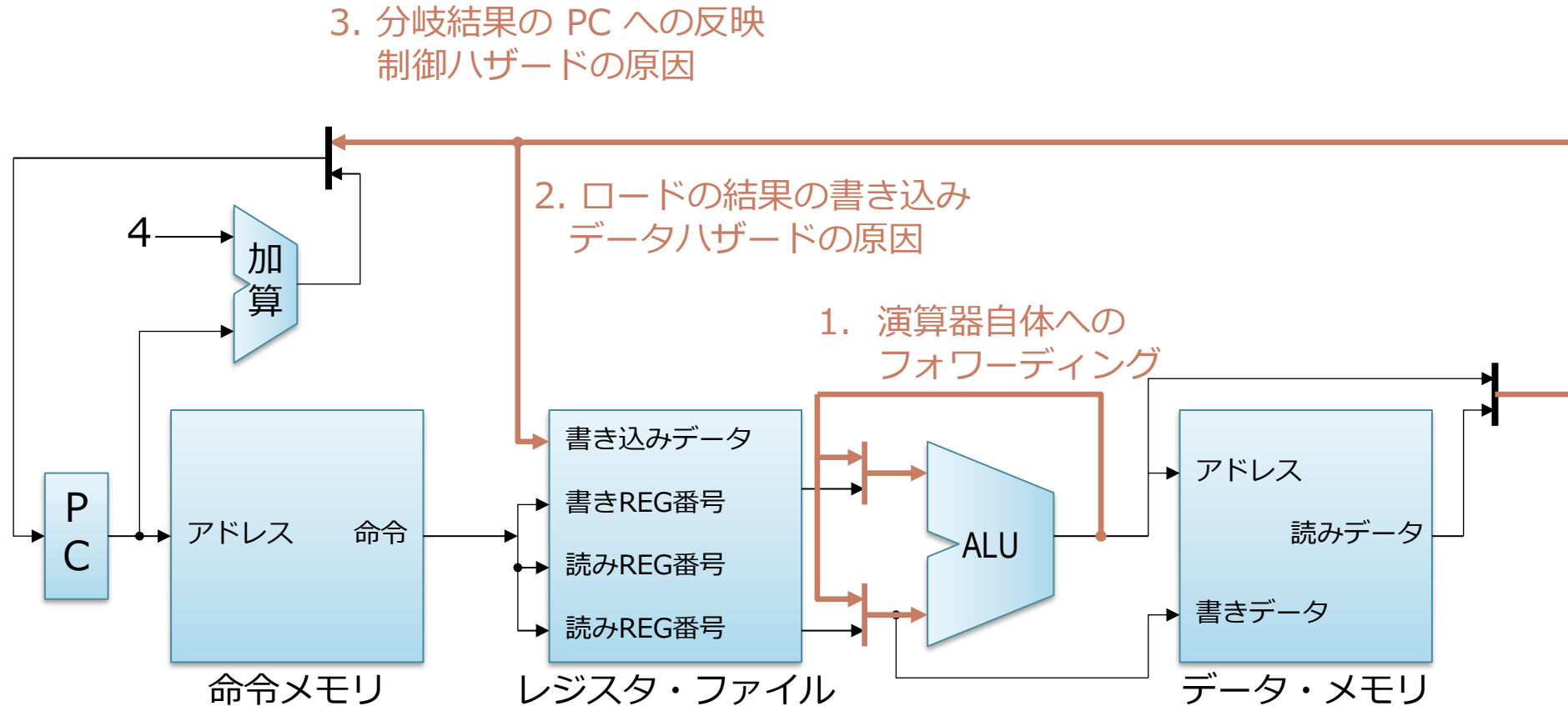
■ CPU の場合

- フォワーディング, 分岐予測, 命令スケジューリング...

■ GPU の場合

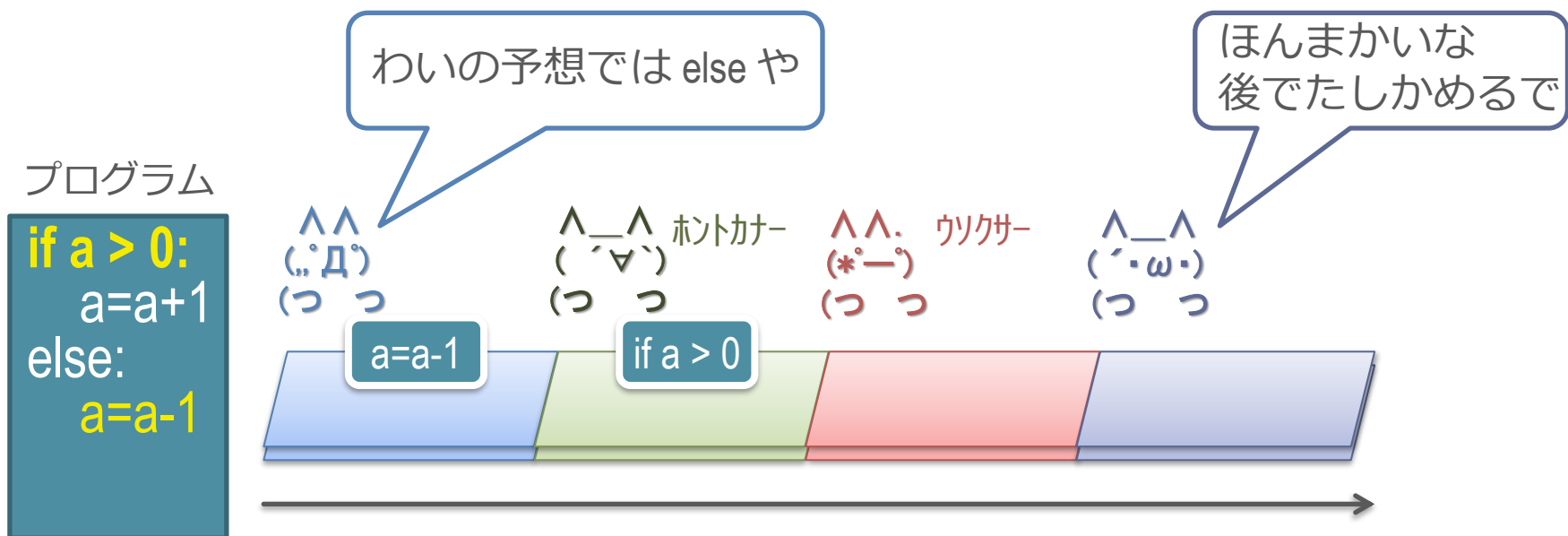
- 上記を一切行わない
- 全部マルチスレッディングで対処
 - 動かすべきスレッドは大量にある

CPU のバックエッジ：逆方向（右から左）



- バックエッジがあるため、命令を単純に流せない場合がある
 - 工場のラインのように、一方向に流せない

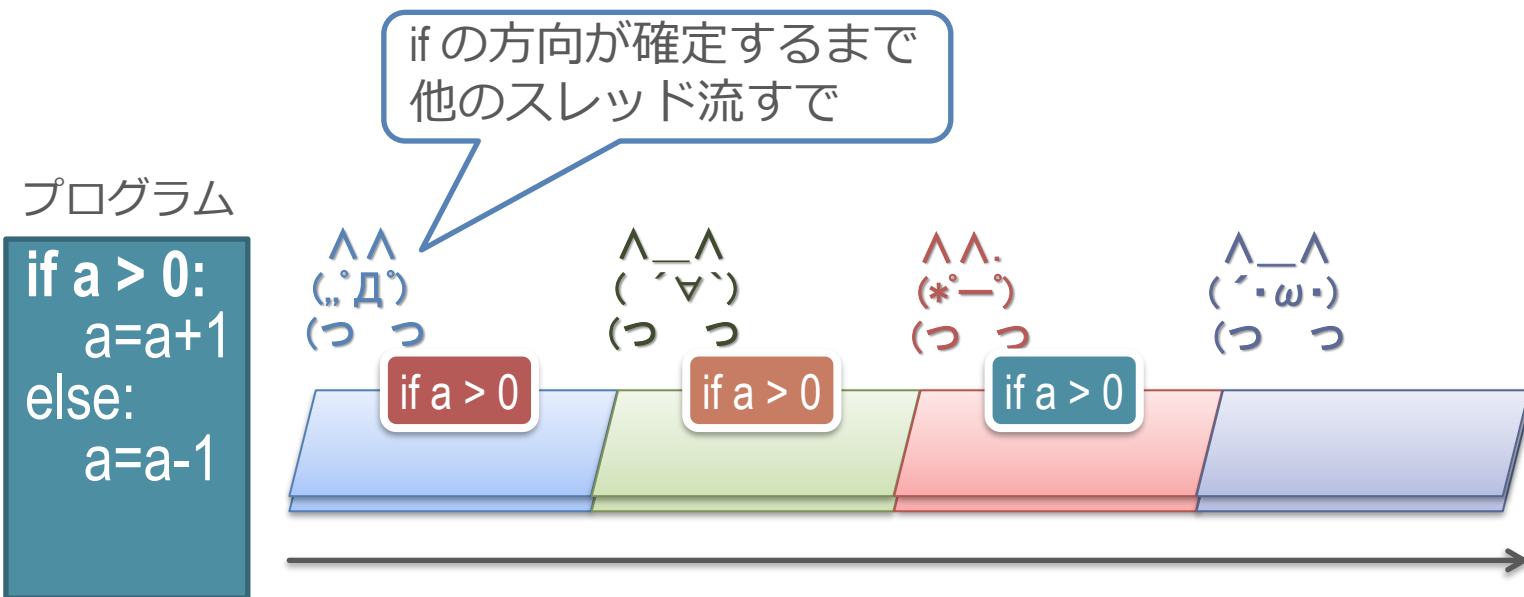
分岐予測



■ 動作

- 「if a > 0」の結果を予測して、命令を取り込む
 - 前はこっちに行ったので、次もこっちに違いないとかで予測
- あとから予測が正しいか確認する

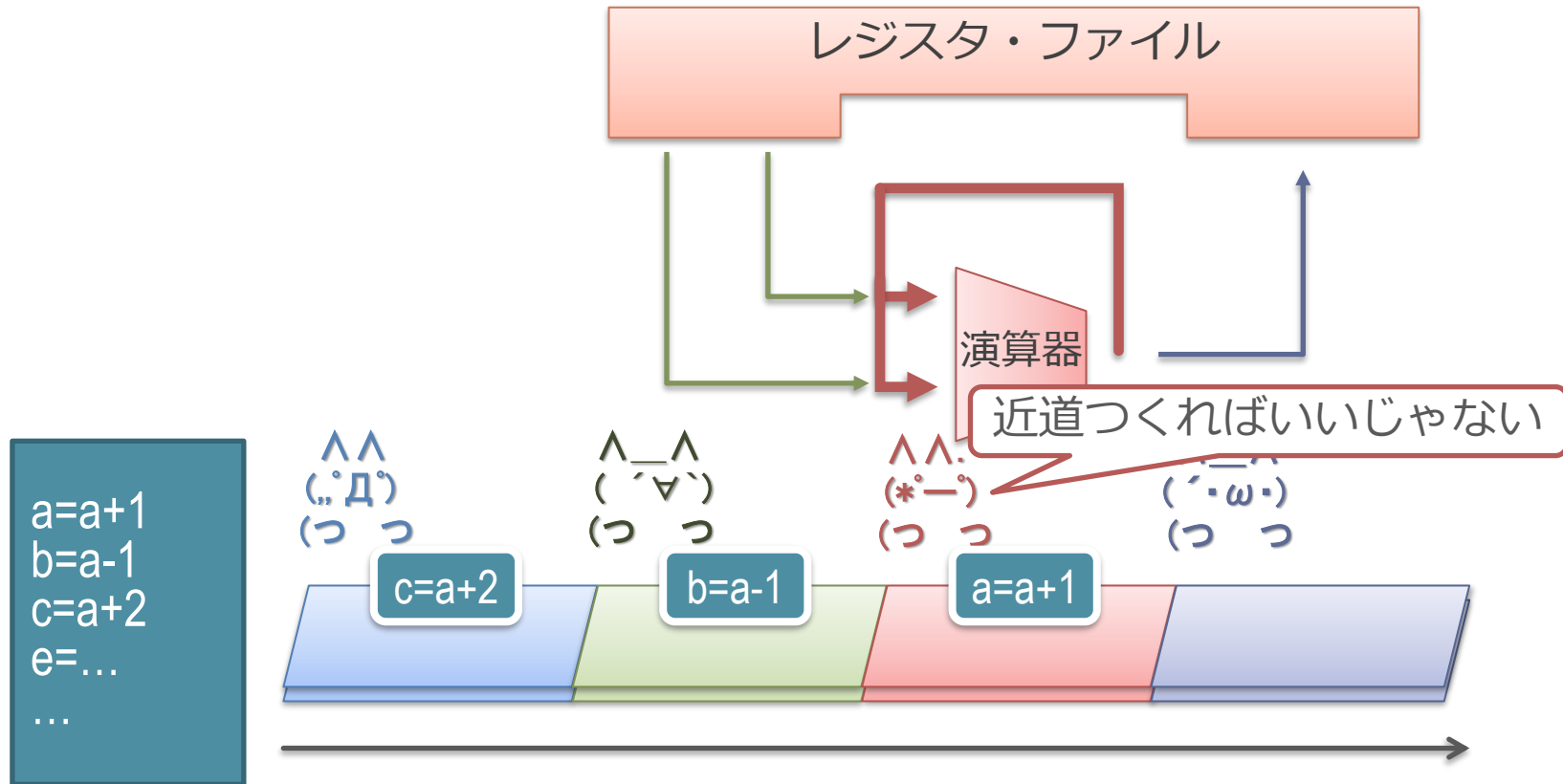
マルチスレッドによる解決



■ 動作

- 「if a > 0」の結果が出るまで他のスレッドの命令をフェッチ
- フェッチすべき候補のスレッドは大量にある

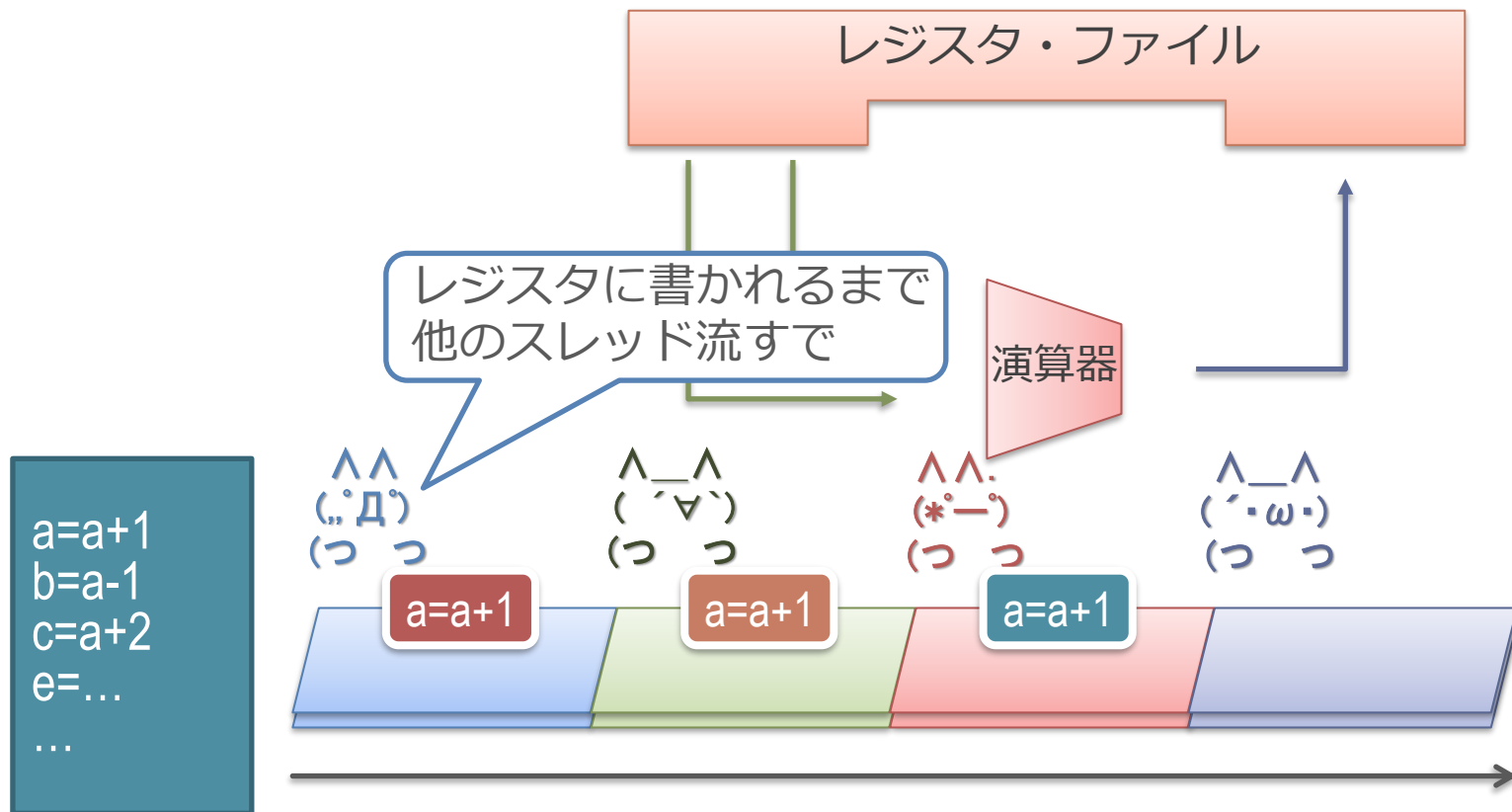
フォワーディング



■ フォワーディング (バイパスとも呼ぶ)

- $(\cdot \cdot -)$ の人が、次のサイクルに結果を使えるようショートカットを作って手元に結果をおいておく

マルチスレッドによる解決



- レジスタ・ファイルに値が書き込まれるまで、他のスレッドを流す
 - 複数のスレッドの a が同時に存在するので、レジスタは大きくなる

今回触れなかった部分の補足

■ 演算の低精度化

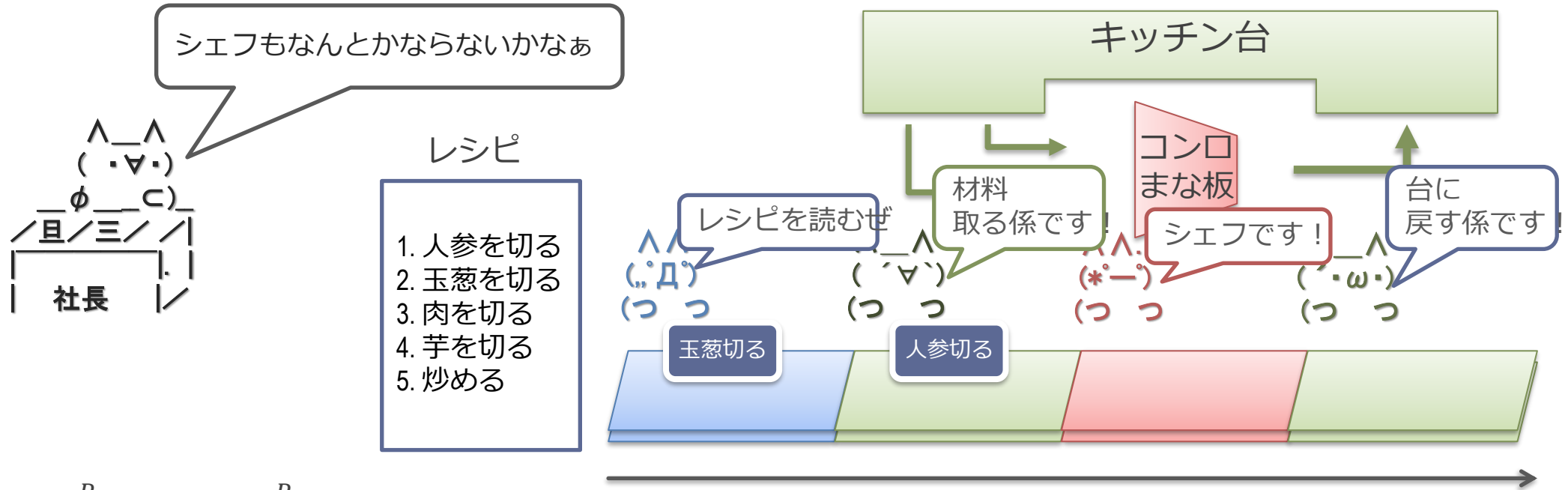
- GPU では演算の精度も低下させて消費エネルギーを削減している

■ マイクロスケーリング (MX)

- 複数の低制度のデータと、共有された指数部から構成
 - 個々の乗算は低精度となり、回路が簡略化される
 - 推論だけでなく、学習でも使用する取り組みが進んでいる
 - なぜ指数を共有できるのか？
 - ◇ 行列演算などでドット積を取ると、最も指数が大きいものが結果に対して支配的になる
 - ◇ 指数が小さいものは精度が低下しても結果への影響が小さい
- MS/ARM/Meta/Intel/AMD/Qualcomm による標準化
 - OCP Microscaling Formats (MX) Specification Version 1.0
 - <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>
- NVIDIA GPU でも Blackwell 世代から採用

(時間がたりないので省略)

カレー屋さんのコストカット



■ 効率 $\eta = \frac{P_{min}}{P_{actual}} = \frac{P_{min}}{P_{min} + P_{overhead}}$

- P_{min} = 材料の原価とシェフの人件費 (絶対カレーを作るのに必要)
- P_{actual} = キッチン台, 他の係の人 (必ずしもなくても)

■ トータルコストを下げる :

- P_{min} を減らす = 切り方を雑にして質を下げて (演算の精度を下げる) トータルコストを下げる
- 見た目の効率 η はこの場合は改善されない ($P_{overhead}$ の相対的な割合が増す)

出欠と感想

- 本日の講義でよくわかったところ，わからなかったところ，質問，感想などを UTOL の出席欄に書いてください
 - LMS の出席を設定するので，そこをお願いします
 - パスワード ： arc
- 意見や内容へのリクエストもあったら書いてください
- UTOL の出席の締め切りは1週間後に設定してあります
 - 仕様上「遅刻」表示になりますが，特に減点等しません

- 以下のスプレッドシートの自分の席のところに学籍番号を書いてください
 - https://docs.google.com/spreadsheets/d/1k83-YtfDNx_Ak40dVgQflaGyh97C2KRbWH-WpvMG5tM/edit?usp=sharing
 - 学生側から見た場合の配置で（画面上が教団側），席の対応する行と列のところに書いてください

- 第11回講義資料を参照

- 最近ではAIが脆弱性を見つけたり、AIが暴走してサイバー攻撃を行ったりと、セキュリティの分野も新しいステージに進んでいると思うので、ハードウェアやOS等の低レイヤーのセキュリティもより難しい分野になって行くのかなというに思いました。

- pectre対策のところの範囲チェックの後にさらにガードを設けるという対策は、事情を知らなければ無駄なコードにも見えますが、原理を踏まえると実は必要だということがわかりました。
- 1 - 2 割性能低下することを許容したとしても、結局攻撃側とのいたちごっこになってしまうのではないかと思った。

- spectreかわいいですね
やっтерことはあまり可愛くなさそうでしたけど、

- よくCを触っていると出てくる「Segmentation Fault」もこの辺りに関係しているエラーなのではないでしょうか（授業内で言及されていたら聞き逃しです、申し訳ありません） "

- 攻撃の話は面白かったが、難しくて4割ぐらいしか理解できなかった。難しい攻撃手法をいっぱい考えるすごい人がいるものだなーと思った。多分紹介されていない攻撃手法とかもいっぱいあるそうだが、ハードウェアやOSを作るときにそれらを全部考慮しないといけないのはすごく大変そうだなと思った。

質問や感想

- ページテーブルの理解があっているか、確認させてください（長いので合っていれば質問回答時間に扱わなくて大丈夫です。間違っている箇所があればその部分を指摘してくれたら幸いです）。
- ・ 32bitアドレス, ページサイズ4KBではページテーブル1エントリに20bit用意しておけば良い。この時, 2段ページテーブルだと, 最大確保要領は $2^{10} + 2^{10} * 2^{10} \doteq 2^{20}\text{bit}$ で単段ページテーブルと変わらないが, 1段目のテーブルでNULLの分は2段目のテーブルが作られないから平均的な確保要領はとても小さくなる。実際, 最小確保要領は（ページが一つは存在するとして, すなわち2段目のページテーブルが1つは存在するとして） $2^{10} + 2^{10} = 2^{11}\text{bit}$ で 2^{20}bit より小さい。
- ・ 再帰的に考えれば, 段数を増やせば増やすほど最小確保要領は小さくなるが, 実際には最小確保要領は平均的なページの数（つまり, 単段ページテーブルにおいてNULLでないエントリの平均的な数）* 20bit を下回る必要はない。というのもどうせ多段だろうが平均的にはこれだけの領域は確保されるから。この段数の範囲で, 最小確保要領と, 物理アクセスを得られるまでの時間とのトレードオフで最適な段数が決まる。

- システムコールは関数呼び出し的にユーザプログラムから呼び出されるが、この形にすることでシステムコールに対応する命令のオペランドがユーザプログラムによって自由に指定されない（OSが管理する）という点にセキュリティ上のメリットがあると考えて良いか？

- またそれを踏まえるとバッファオーバーフローのスライドp.65で「外部プログラムを起動するシステムコールを呼ぶ」とあるが、正しくは「システムコールに対応する命令」ではないのか？（自由にオペランドを設定できるため、権限のあるプロセスを外部プログラムに置き換え、外部プログラムのプロセスに権限を持った状態にできてしまう、ということだと思いました）（システムコールは、呼ぶだけならユーザプログラムからいつでも呼べるという理解をしています）

- Spectreの話でいろいろ対策案があったのですが、今のところのどの対処法でも性能が低下してしまっているのですが、こういった脆弱性の対策と性能は完全なトレードオフになっているのでしょうか？

- おばけしか見てなくて、枝を持っていることに気付いてなかった

- 近年では、Rustのようなメモリ安全性を保証する言語がOSやシステムソフトウェアにも徐々に導入されていると認識していますが、今後バッファオーバーフローなどの脆弱性はどの程度減少すると期待できるのでしょうか。個人的に自分がやっている分野はまだC/C++が多い(特に脆弱性などは気にせず)のでRustを使う必要性まで特に感じてはないですが。

- Spectre Variant 1で、`array1\_size`を意図的にキャッシュミスさせて分岐の確定を遅らせ、その間に範囲外アクセスをOoO実行させるとのことでしたが、実際のCPUではこの投機実行の時間幅を攻撃者がどの程度制御できるのでしょうか。

- 論理メモリと物理メモリのページ番号などはパタヘネで初めて勉強したときは酷く混乱した記憶があります そしてTLBが導入されてもっと複雑に... 確かそのときはメモリアクセスの全パターン(キャッシュ→TLB→ページテーブル、でいいんですかね)を書き出して勉強した記憶があります

- ソフトウェアによる対策が開発された後にもハードウェアによる Spectreの対策が研究されているのは、やはり前者ではオーバーヘッドが大きいいためなのか、およびハードウェアによる対策への期待感が実際のところどの程度なのかが気になりました。

- あと、spectre という命名（直訳すると幽霊？）については、007シリーズファンとしては（映画内では、ジェームズボンドの天敵として再三登場する影の悪の組織みたいな存在です）通ずるものを感じ、親近感が湧いています。きっとマイクロアーキテクチャの研究者にとってはそのような存在なんではないでしょうか。